

Advance Health Monitoring System

ECG and Heart Rate Analysis for Early Detection and Diagnosis

Ashutosh Rajendra karve

Bits-ID: **2024ht01021**

2024ht01021@wilp.bits-pilani.ac.in

+91 9765541324

Pune, Maharashtra

14/09/2024

<< [GitHub Project Link](#) >>

clone: https://github.com/Ashutoshkarve007/HealthMonitoring_BITS_Pilani_2024ht01021.git

GUIDED FOR DETAILED UNDERSTANDING OF ECG AND WORKING

Dr. Prachi Pawar (BHMS)

Reg No:(85206)

Pune, Maharashtra

Abstract

This project develops an advanced health monitoring system that uses the STM32 Blue Pill microcontroller and AD8232 ECG sensor to capture comprehensive cardiovascular data. The system is designed not only to monitor heart rate but also to detect critical conditions such as heart attacks, myocardial infarction, and electrolyte imbalances through sophisticated signal processing and diagnostic algorithms implemented in Python. Data captured in real-time is analyzed to identify abnormal rhythms and potential cardiovascular issues, providing detailed health reports within minutes.

1 Introduction

The **Advance Health Monitoring System** is a project aimed at developing a real-time ECG monitoring solution using the STM32 Blue Pill microcontroller and AD8232 ECG sensor. The primary objective of this project is to capture ECG data from a user, transmit it via USB to a Python-based system for analysis, and visualize the ECG signal. The current version of the project is focused on raw ECG data acquisition and visualization, with plans for further development, such as AI-based analysis, in the future.

2 System Design

The system is designed to acquire ECG data from a human subject using a combination of hardware and software components:

Hardware:

- **STM32 Blue Pill Microcontroller:** Used for data acquisition via its analog-to-digital converter (ADC) and communication over USB (virtual COM port).
- **AD8232 ECG Sensor:** Measures the electrical activity of the heart and outputs analog signals, which are fed into the STM32 for processing.

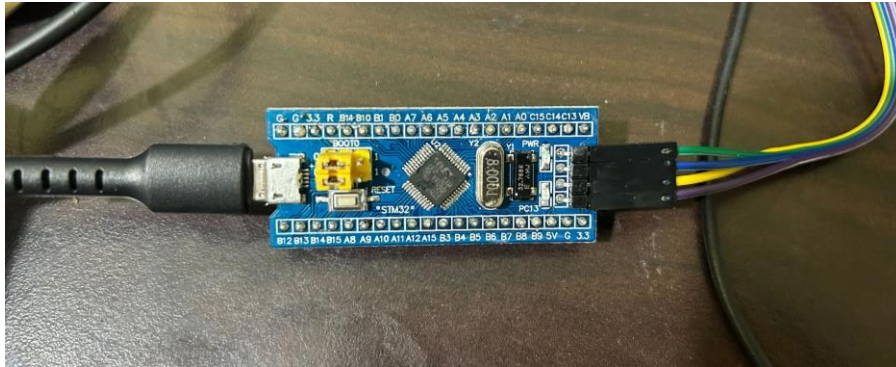
Software:

- **STM32 Firmware:** The firmware reads raw ADC data from the ECG sensor connected to the STM32's PA1 (A1) pin. The raw data (0-4095 for 12-bit ADC) is transmitted to a PC via USB.
- **Python Interface:** The Python code reads the raw ECG data from the STM32 and plots the real-time ECG signal. The data is visualized to observe the heart's electrical activity over time.

3 Hardware Integration

Step 1: Assembling the Hardware

- **Components Required:**
 - STM32 Blue Pill microcontroller



STM32(Blue Pill) Development Board

Microcontroller:

Part	:	STM32F103C8T6
Manufacturer	:	ST-Microelectronics
Core	:	Arm Cortex-M3
Max. Clock	:	Speed 72MHz
Package	:	LQFP 48 pins

Internal memories:

FLASH	:	64KiB
SRAM	:	20KiB

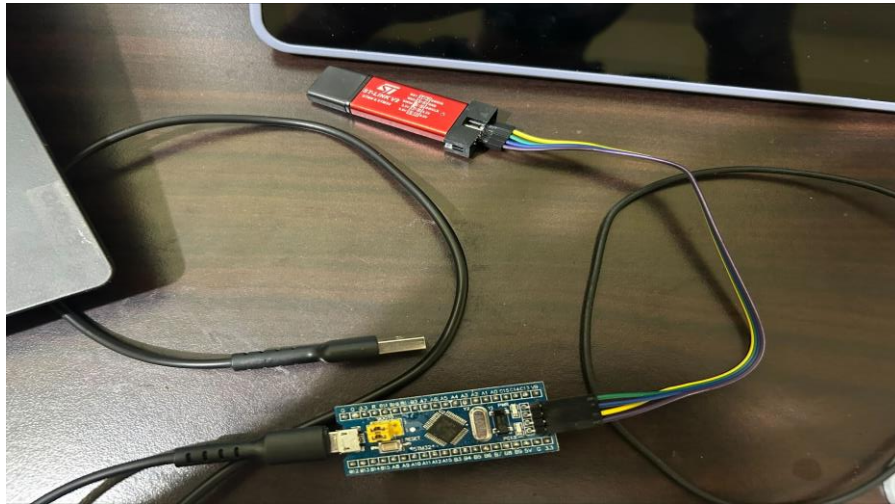
Oscillators:

HSI	:	8MHz
LSI	:	40kHz
HSE	:	8MHz
LSE	:	32.768kHz

Power:

Sources	:	Any +3.3V pin (+3.3V) Any +5V pin (+5V) USB connector (+5V)
---------	---	---

- USB cables & ST Link V2 for Programming



Possible Connections

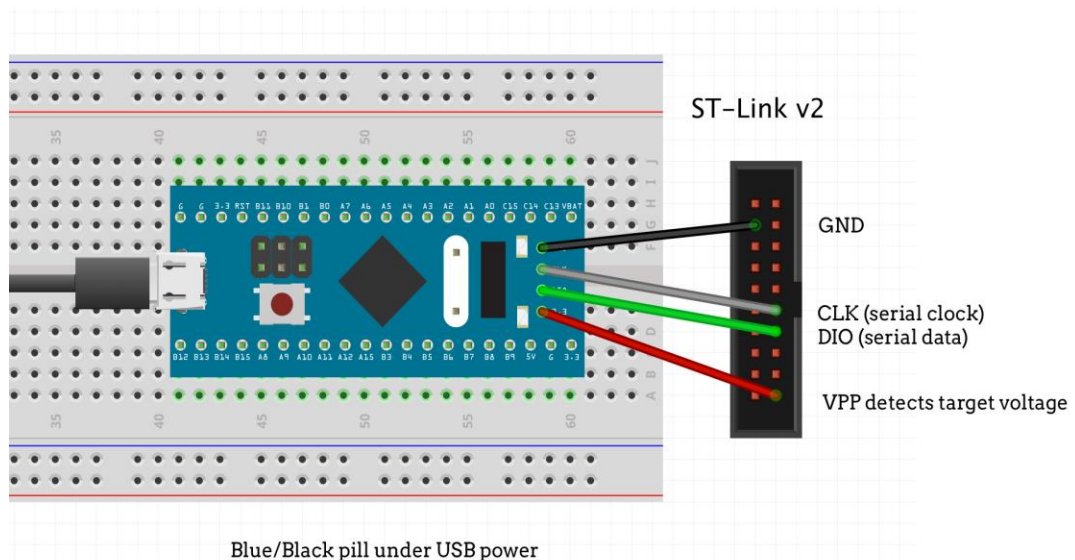
To use a ST-Link to program a blue/black pill (BP) under external power, connect it like this:

ST-Link GND (even-numbered pins from 4 to 20) to BP Ground

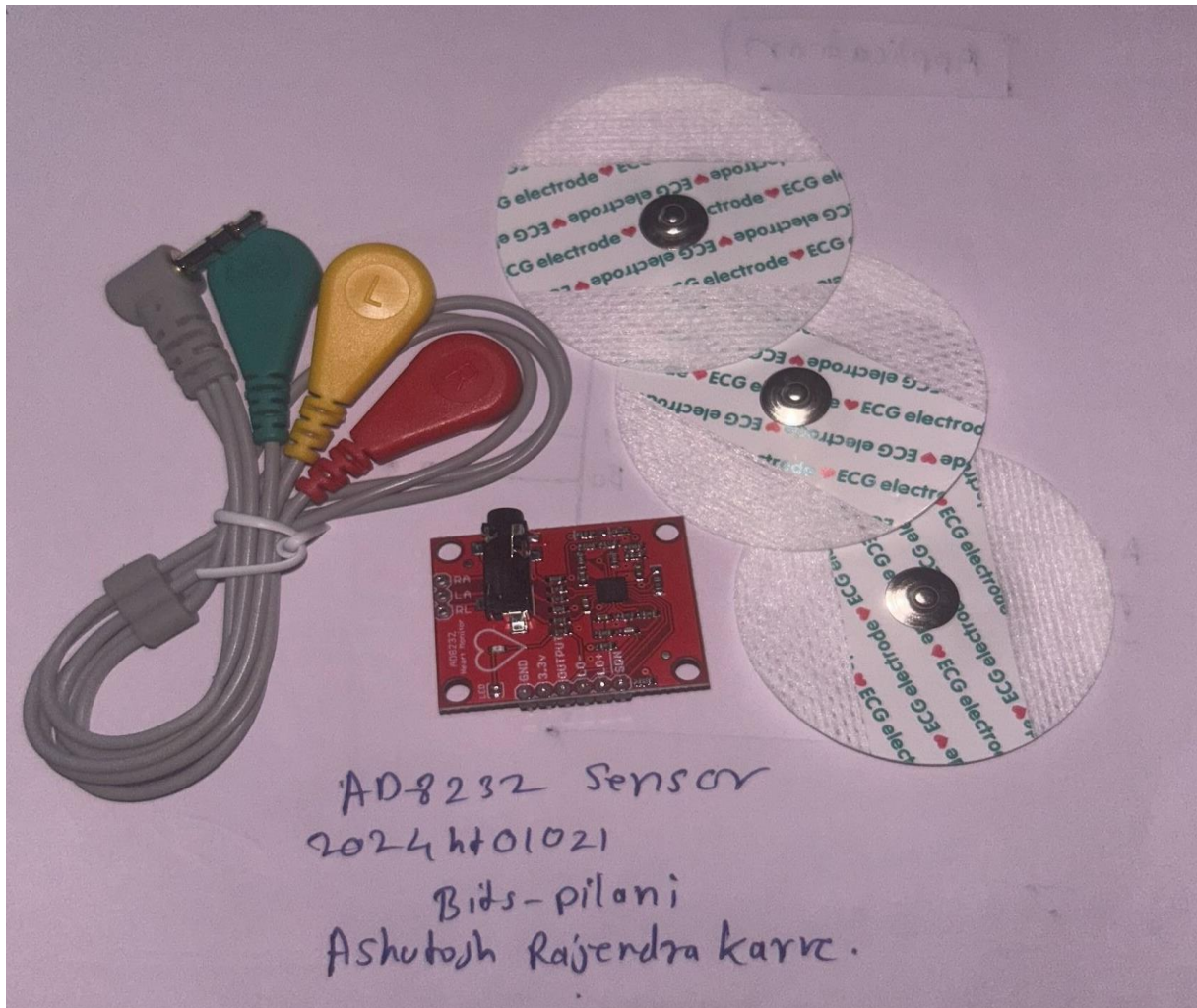
ST-Link SWDIO (pin 7) to BP DIO

ST-Link SWCLK (pin 9) to BP CLK

ST-Link VAPP (pins 1 or 2) to BP 3V3



- AD8232 ECG Sensor



The **AD8232 ECG sensor** is designed for measuring heart activity by capturing, amplifying, and filtering ECG signals. It removes noise and interference, providing a clean analog signal for further processing.

Key Features:

- **Low Power:** Ideal for portable health monitoring.
- **Noise Filtering:** Removes motion artifacts and electrical interference for cleaner signals.
- **Electrode Inputs:** Requires three electrodes (right arm, left arm, right leg) for accurate readings.
- **Analog Output:** Provides analog ECG signals to the STM32's ADC for digitization.

In this project, the sensor's output is connected to **PA1 (A1)** on the STM32, which digitizes and transmits the ECG data over USB for real-time analysis and visualization.

- Heartbeat Pulse Sensor (For Future scope)



The **Heartbeat Pulse Sensor** is designed to measure heart rate by detecting the blood flow through the skin using photoplethysmography (PPG). It provides an easy way to monitor pulse activity by converting the changes in light absorption, caused by blood flow, into an electrical signal.

- Connecting wires

- **Procedure:**

1. **Power Connections:**

- Connect the 3.3V and GND pins of the STM32 to the respective pins on the AD8232 and Heart Pulse Sensor.

2. **Signal Connections:**

- Connect the OUTPUT of the AD8232 to PA1 on STM32 for ECG data capture.
- Connect the OUTPUT of the Heart Pulse Sensor to PA0 on STM32 for heart rate measurement. (Future Scope)

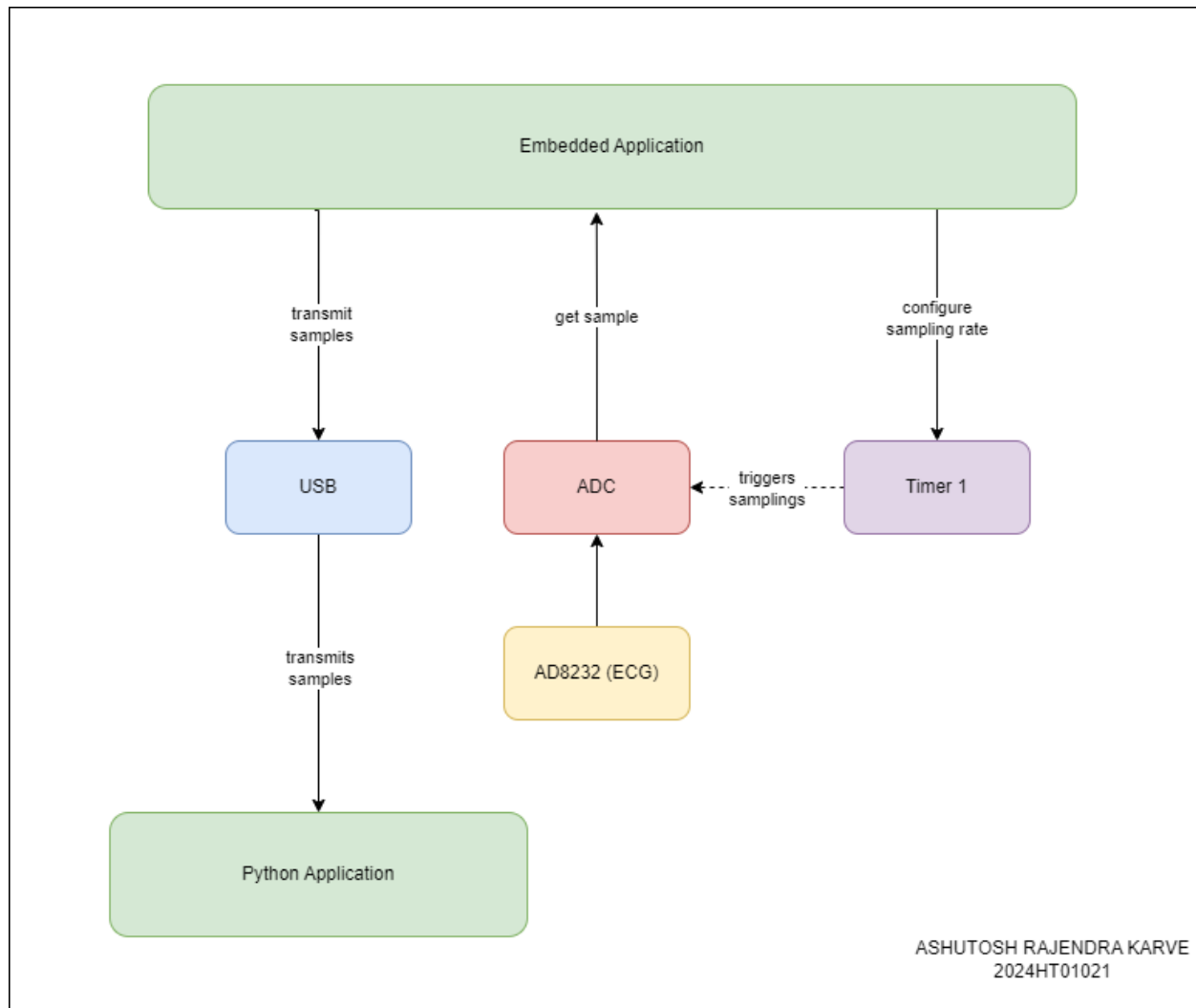
3. **USB Configuration:**

- Ensure PA11 (USB_DM) and PA12 (USB_DP) are connected for USB functionality.

4. **Configuring Leads-Off Detection:**

- Use PB0 and PB1 on STM32 to connect to LO- and LO+ on the AD8232 for detecting if the leads are properly attached.

- **Hardware Architecture**

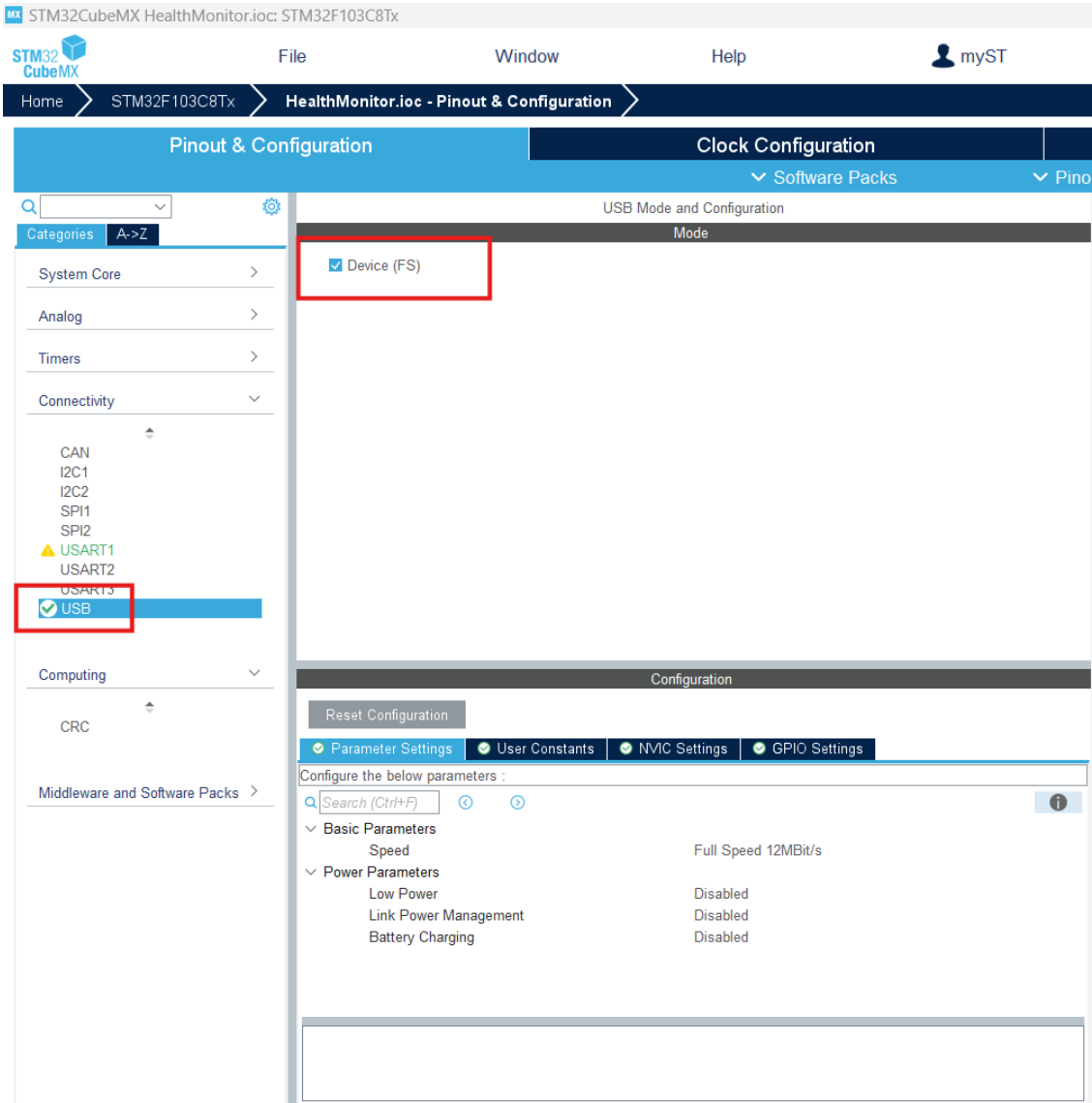


The application was developed using Keil MDK Software for stm32f103c8 microcontroller. And pin configuration was done on STM32 CUBEMX, we are collecting data from AD8232 sensor from ADC to STM32F103 and then data is transmitted to any PC/Raspberry pi.

Step 2: STM32 CUBE MX Pin Configuration

- **USB Configuration on STM32 Blue pill**

I will use the STM32F103C8 controller in the USB DEVICE mode, and the Communication Device class (Virtual Com Port). The setup for the same is shown below screen shot



First, I am selecting the USB in DEVICE only MODE, as shown above

STM32CubeMX HealthMonitor.ioc: STM32F103C8Tx

File Window Help myST

Home > STM32F103C8Tx > HealthMonitor.ioc - Pinout & Configuration

Pinout & Configuration Clock Configuration

Software Packs Pinout

Search

Categories A-Z

CRC

Middleware and Software Pac...

AIROC-Wi-Fi-Bluetooth-STM...
FATFS
FP-SNS-MOTENVWB1
FP-SNS-SMARTAG2
FP-SNS-STBOX1
FREERTOS
I-CUBE-Cesium
I-CUBE-FS-RTOS
I-CUBE-ITTIADB
I-CUBE-Mongoose
I-CUBE-embOS
I-CUBE-wolfMQTT
I-CUBE-wolfSSH
I-CUBE-wolfSSL
I-CUBE-wolfTPM
I-Cube-SoM-uGOAL
USB_DEVICE
X-CUBE-ALGOBUILD
X-CUBE-ALS
X-CUBE-BLE1
X-CUBE-BLE2
X-CUBE-BLEMGR
X-CUBE-DPower
X-CUBE-EEPROMA1
X-CUBE-GNSS1
X-CUBE-ISPU
X-CUBE-MEMS1
X-CUBE-NFC4
X-CUBE-NFC6
X-CUBE-NFC7
X-CUBE-RT-Thread_Nano
X-CUBE-SAFE1
X-CUBE-SFXS2LP1
X-CUBE-SMBUS
X-CUBE-SUBG2
X-CUBE-TCP
X-CUBE-TOF1

USB_DEVICE Mode and Configuration

Mode

Class For FS IP Communication Device Class (Virtual Port Com)

Configuration

Reset Configuration

Parameter Settings Device Descriptor User Constants

Configure the below parameters :

Search (Ctrl+F)

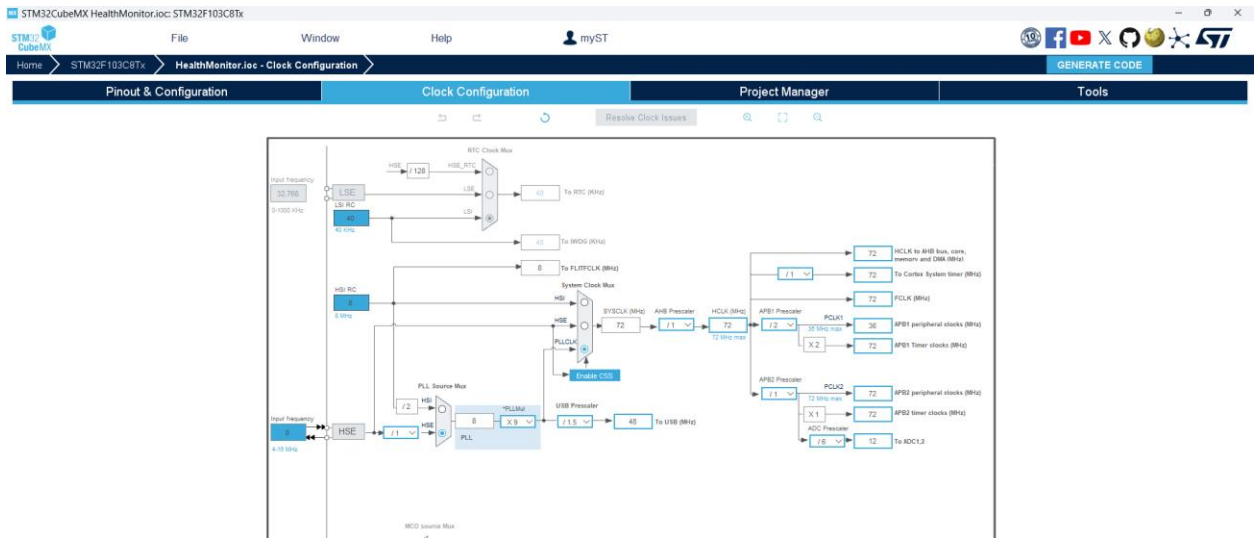
Basic Parameters

USBD_MAX_NUM_INTERFACES (Maximum nu...	1
USBD_MAX_NUM_CONFIGURATION (Maximum...	1
USBD_MAX_STR_DESC_SIZ (Maximum size for...	512 bytes
USBD_SELF_POWERED (Enabled self power)	Enabled
USBD_DEBUG_LEVEL (USBD Debug Level)	0: No debug message

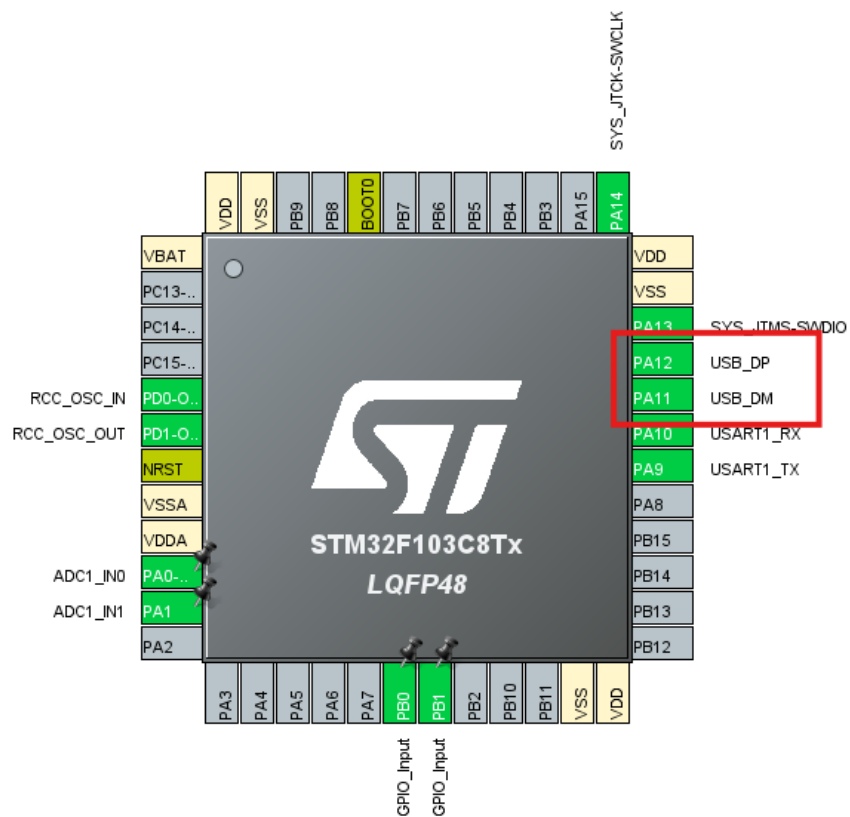
Class Parameters

USB CDC Rx Buffer Size	1024 Bytes
USB CDC Tx Buffer Size	1024 Bytes

in the USB Device, Select the class as Communication Device Class (Virtual Port Com)

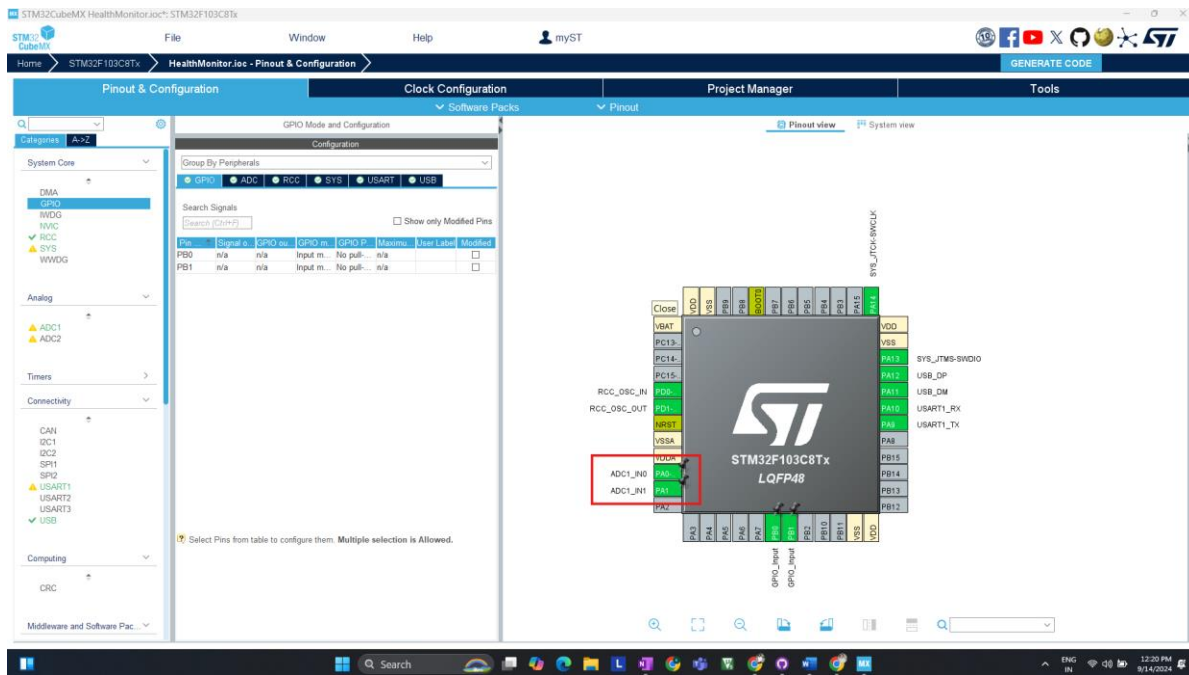


Finally, the clock is set to maximum here. As you can see above, the USB clock is automatically adjusted to 48 MHz

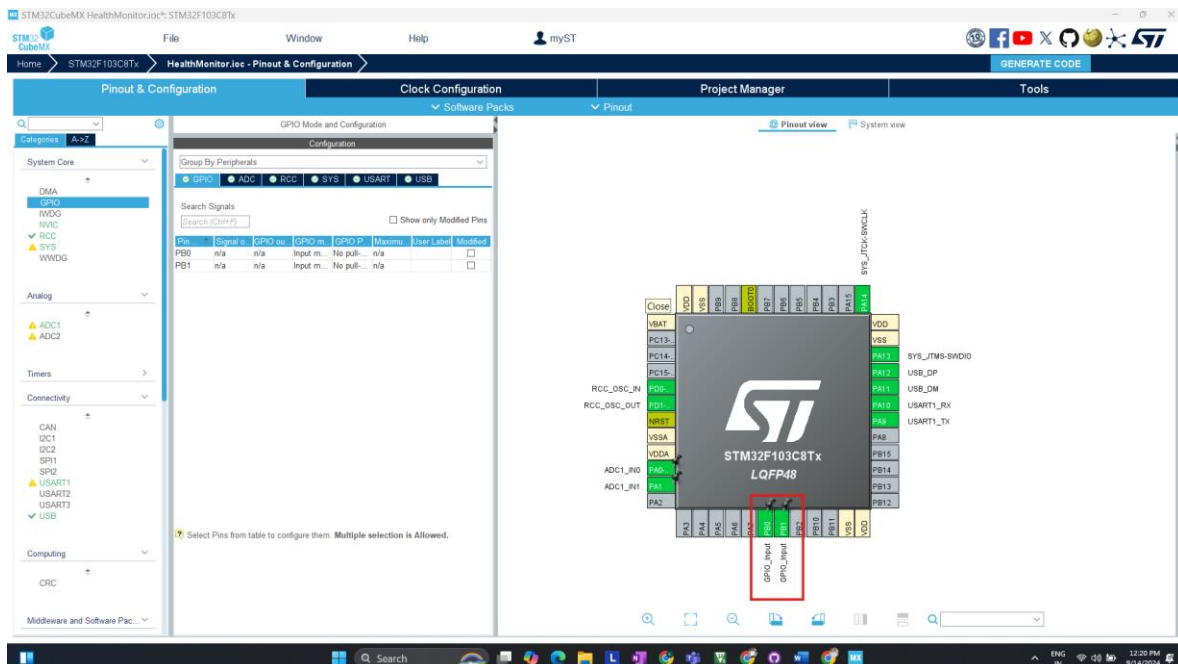


Ensure PA11 (USB_DM) and PA12(USB_DP) are connected for USB functionality

- **Sensor connection and configuration:**



ADC Configuration is done for AD8283 & Heart Pulse detector sensor, AD8232 to **PA1** on STM32 for ECG data capture, Heart Pulse Sensor to **PA0** on STM32 for heart rate measurement.



Use PB0 and PB1 on STM32 to connect to LO- and LO+ on the AD8232 for detecting if leads are properly attached.

4 Software Integration (Firmware Development & Testing STM32)

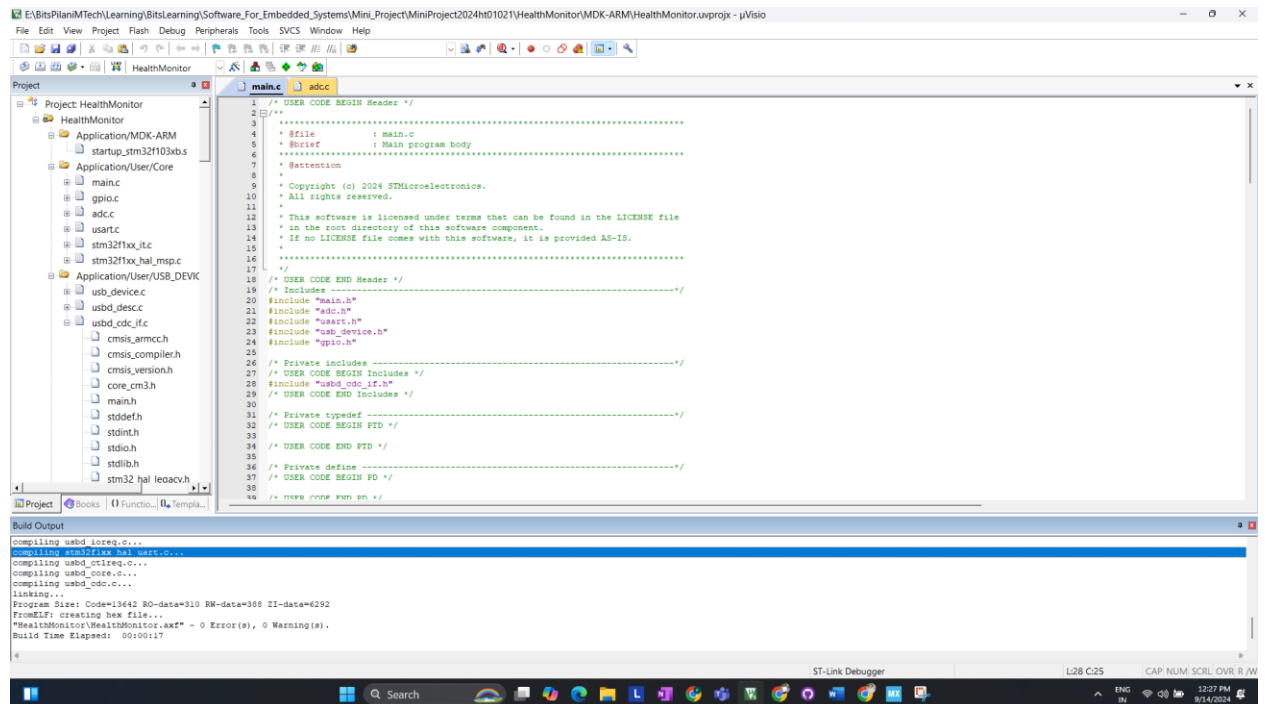
Step 1: Environment Setup

- **Tools Required:**

- Keil V5 for programming and debugging.

- **Procedure:**

1. Install Keil V5 and configure the project for STM32F103C8Tx.
2. Set up project directories and necessary libraries (HAL, USB CDC).

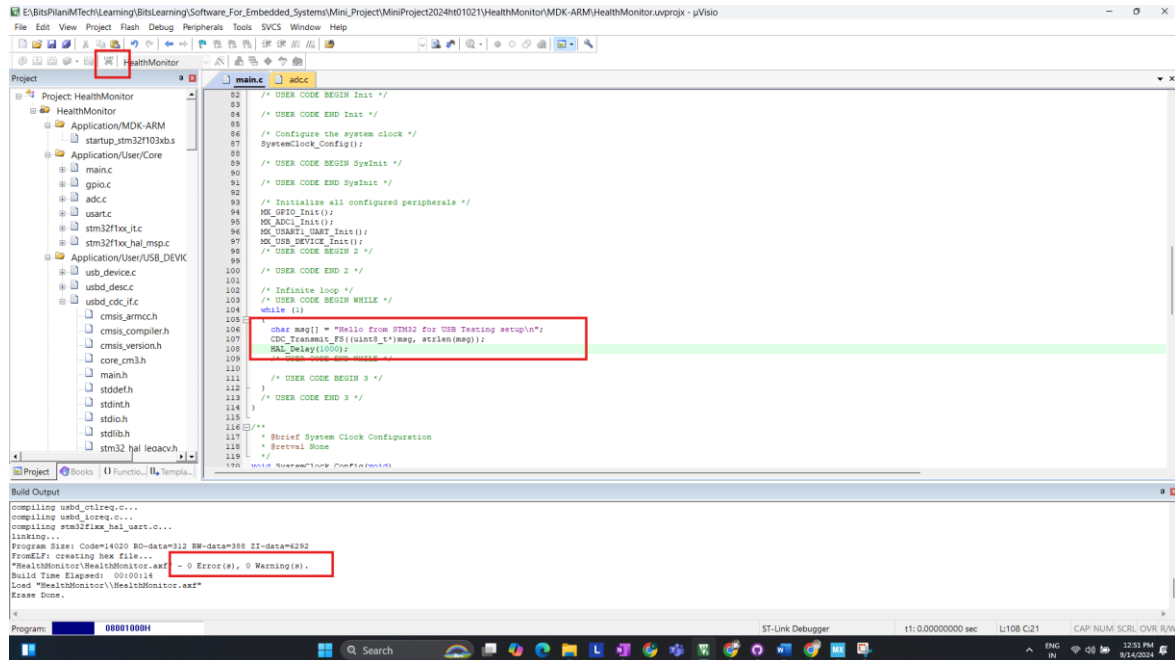


Screen shot of error less code after generating code from STM32 Cube MX.

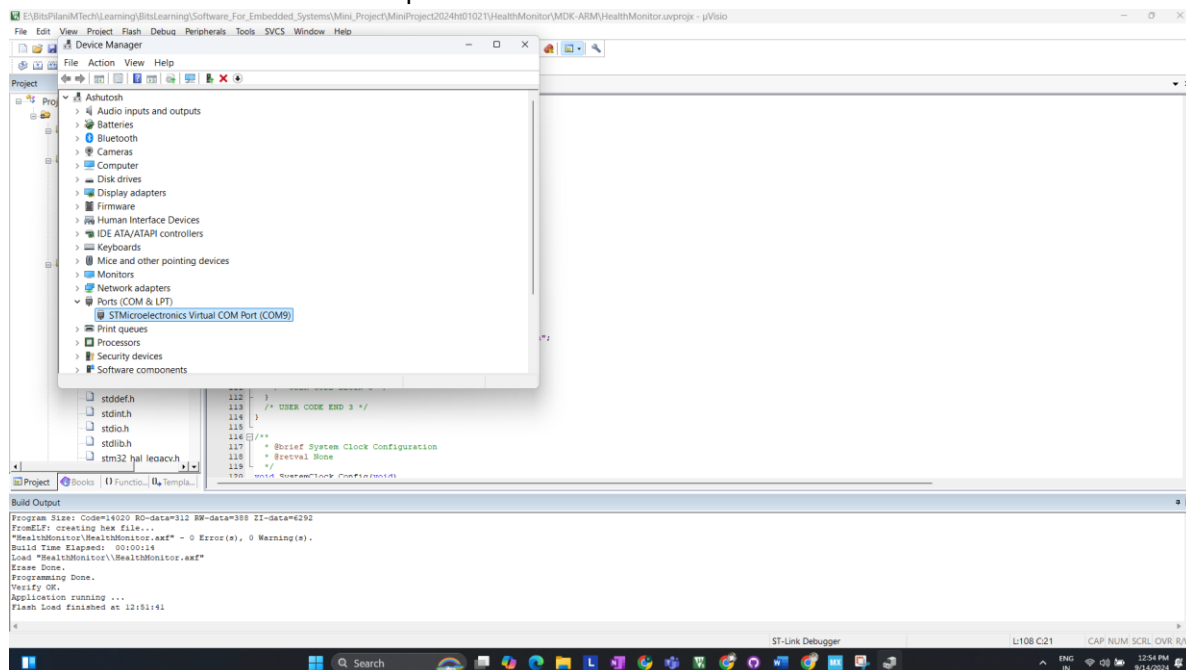
Step 2: Coding (Initial Basic Setup & Testing)

1. USB Communication Setup:

Implement USB CDC for data transmission to Ubuntu.



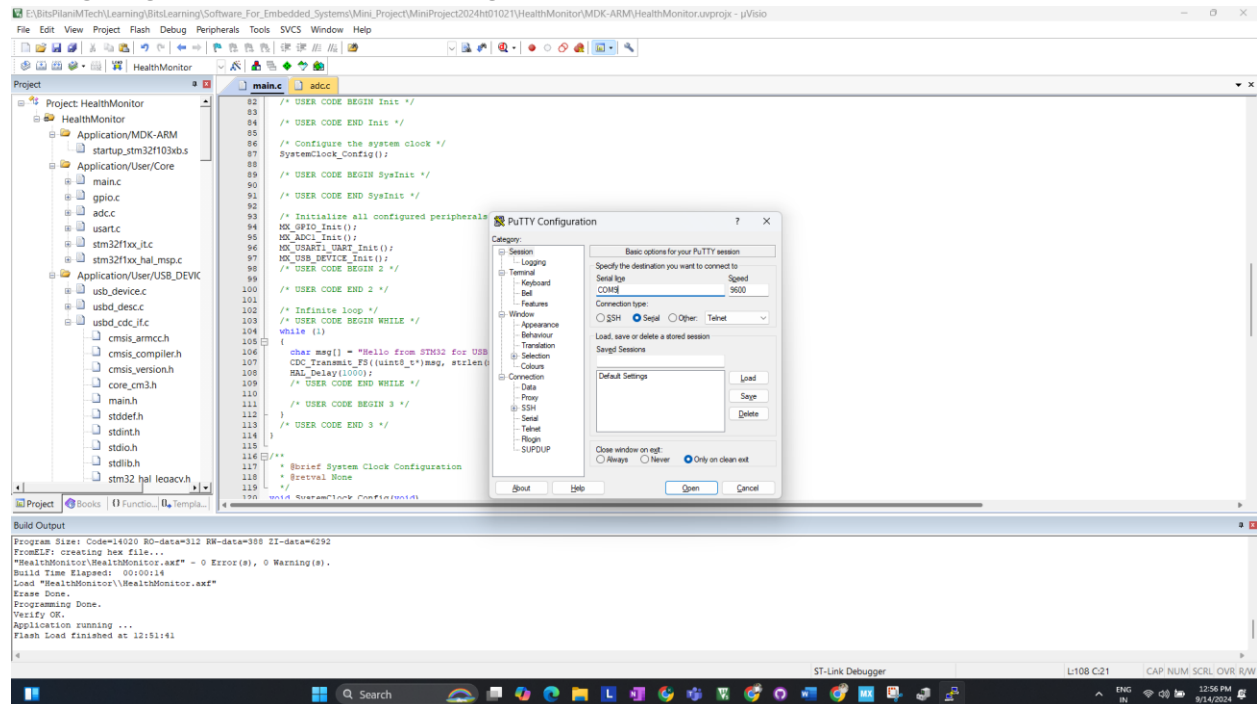
In above screenshot I have compiled and flash code for USB CDC.



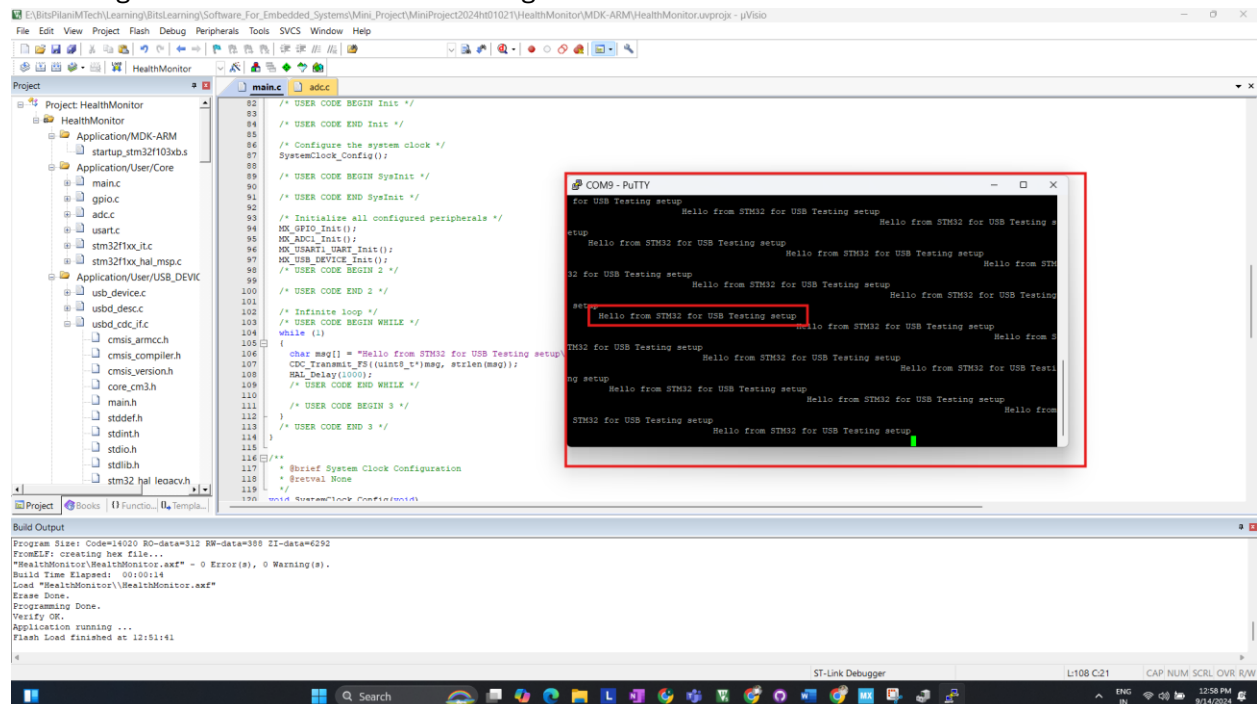
We can detect COM Port (COM9) for Our device (STM32 Blue Pill)

2. Checking with putty:

Configuring Serial to COM9 and setting baud rate to 9600

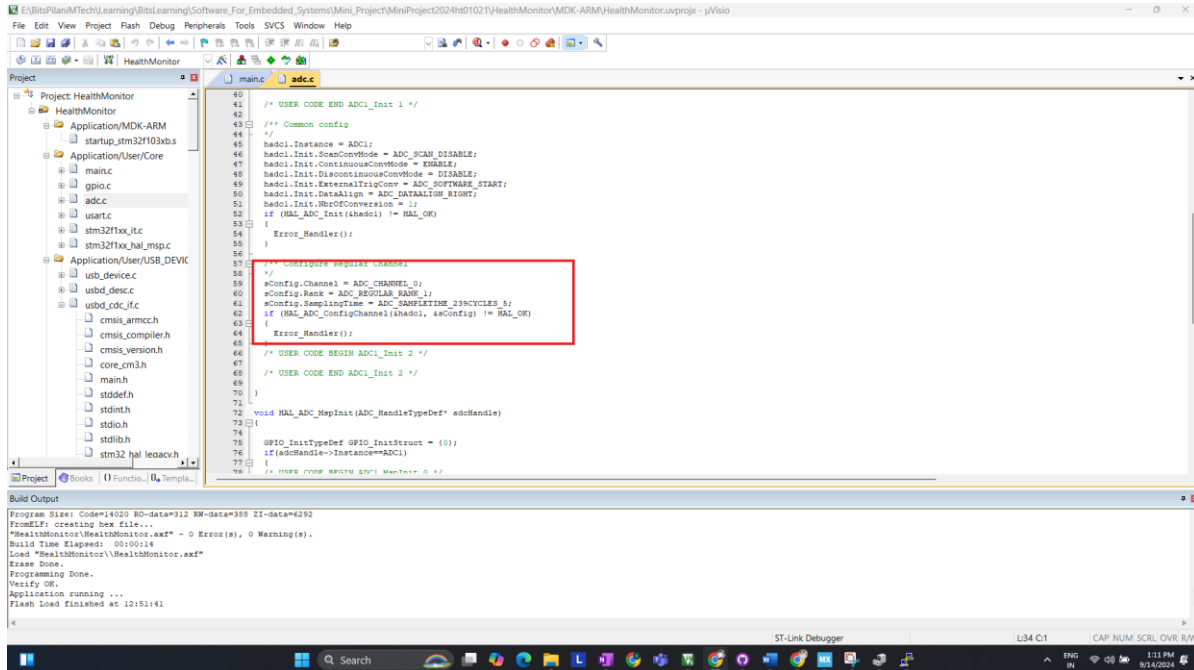


Checking Serial Terminal We are receiving the DATA



3. ADC Configuration:

The ADC is configured to read analog values from the ECG sensor connected to PA1 and the Heart Pulse sensor connected to PA0. The following code snippet demonstrates how to configure the ADC

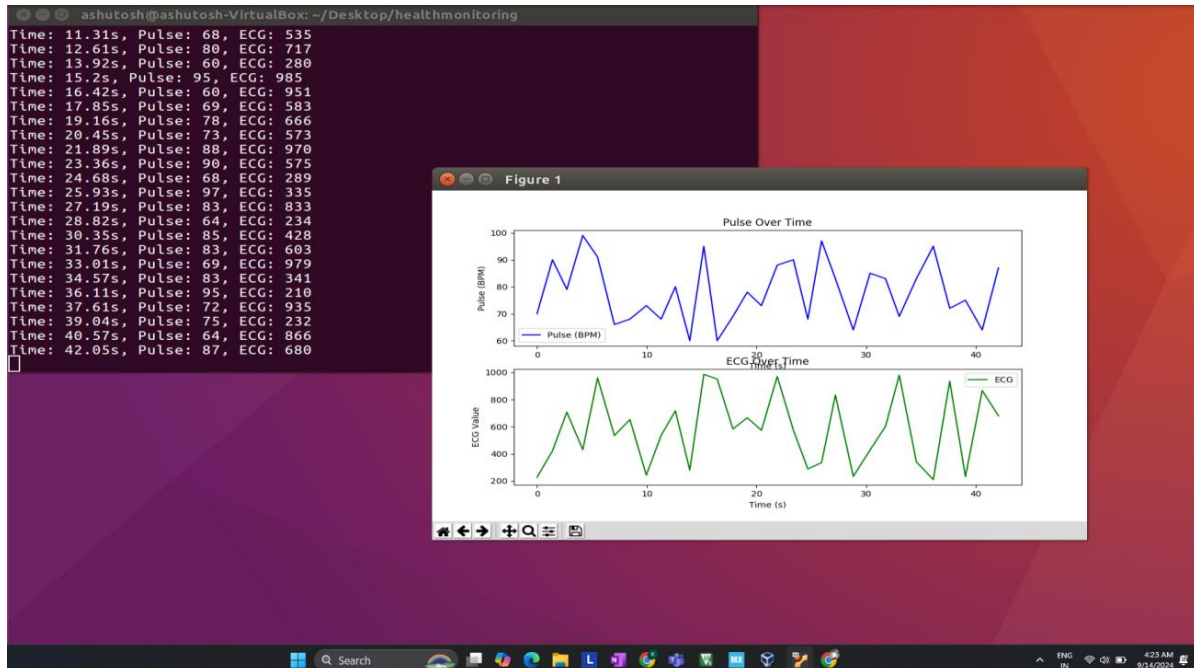


The screenshot shows an IDE with a project named 'HealthMonitor'. The 'main.c' file is open, displaying ADC configuration code. A red box highlights the initialization of the ADC channel and the start of the first ADC conversion. The build output at the bottom shows the program size and successful compilation.

```
40 /* USER CODE END ADC_Init 1 */
41
42 /** Common config
43 */
44 hadci.Instance = ADC1;
45 hadci.Init.ScanConvMode = ADC_SCAN_DISABLE;
46 hadci.Init.ContinuousConvMode = ENABLE;
47 hadci.Init.DiscontinuousConvMode = DISABLE;
48 hadci.Init.ExternalTrigConv = ADC_SOFTWARE_START;
49 hadci.Init.DataAlign = ADC_DATAALIGN_RIGHT;
50 hadci.Init.NbrOfConversion = 1;
51 if (HAL_ADC_Init(&hadci) != HAL_OK)
52 {
53     Error_Handler();
54 }
55
56 /** Configure regular channel
57 */
58 sConfig.Channel = ADC_CHANNEL_0;
59 sConfig.Rank = ADC_REGULAR_RANK_1;
60 sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
61 if (HAL_ADC_ConfigChannel(&hadci, &sConfig) != HAL_OK)
62 {
63     Error_Handler();
64 }
65 /* USER CODE BEGIN ADC_Init 2 */
66 /* USER CODE END ADC_Init 2 */
67
68 void HAL_ADC_MspInit(ADC_HandleTypeDef* adcHandle)
69 {
70     GPIO_InitTypeDef GPIO_InitStruct = {0};
71     if(adcHandle->Instance==ADC1)
72     {
73         /* GPIO configuration for ADC1 */
74     }
75 }
76
77 /* USER CODE BEGIN ADC_MspInit 0 */
78 /* USER CODE END ADC_MspInit 0 */
```

Build Output
Program Size: Code=14020 RO-data=312 RW-data=388 ZI-data=4292
FromELF: creating hex file...
"HealthMonitor\HealthMonitor.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:14
Load "HealthMonitor\HealthMonitor.axf"
Erase Done.
Programming Done.
Verify OK.
Application running ...
Flash Load Finished at 12:51:41

4. Testing and debugging on Ubuntu



Initial Fetching of RAW DATA for ECG and PULSE ON UBUNTU

Step 3: Software Development on Ubuntu for Data Processing (Testing)

After completing the firmware development and testing the STM32's ability to transmit data via USB, the next step involves setting up the Python-based system on Ubuntu.

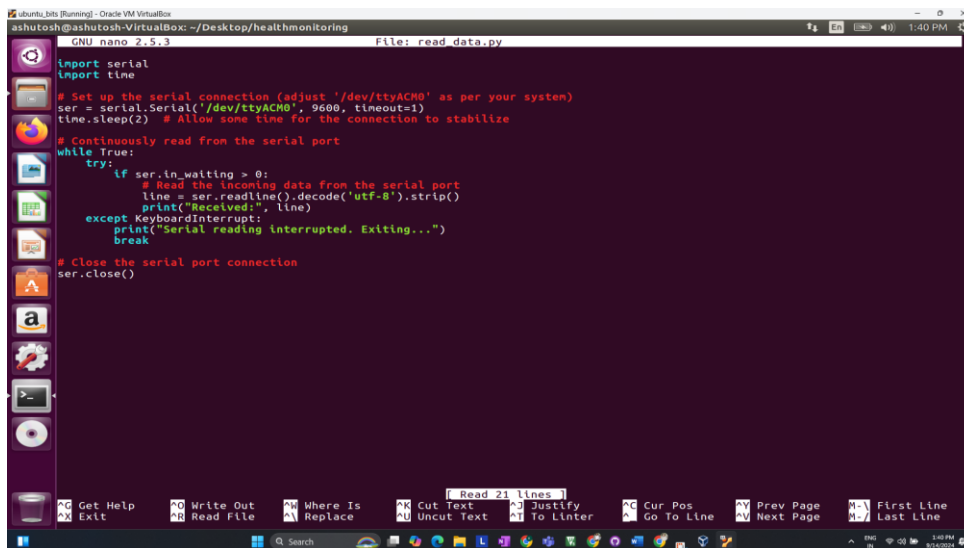
1: Setting Up the Python Environment

Tools Required:

- Python 3.6 or higher
- Libraries:
 - **pyserial** for USB communication
 - **matplotlib** for visualizing ECG and pulse data
 - **numpy** for data handling and signal processing
 - **scipy** (optional) for more advanced signal processing and peak detection

2: Writing Python Code to Read Data from STM32

The Python script will connect to the STM32 via the virtual COM port over USB, read the incoming data, and then process it. Below is a simple script to start reading the data. (Basic program to check connection with STM32 USB CDC)



```
ashutosh@ashutosh-VirtualBox: ~/Desktop/healthmonitoring
GNU nano 2.5.3 File: read_data.py

import serial
import time

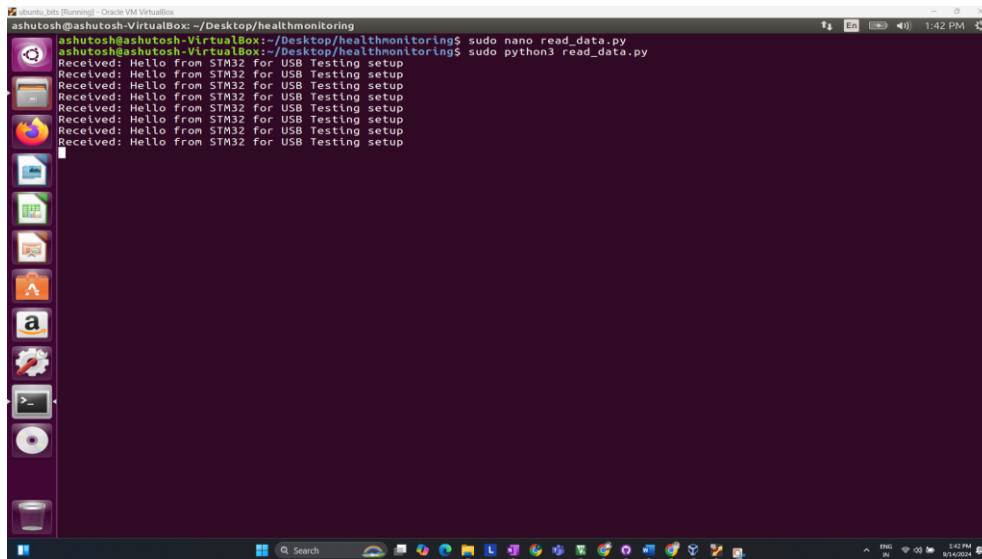
# Set up the serial connection (adjust '/dev/ttyACM0' as per your system)
ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
time.sleep(2) # Allow some time for the connection to stabilize

# Continuously read from the serial port
while True:
    try:
        if ser.in_waiting > 0:
            # Read the incoming data from the serial port
            line = ser.readline().decode('utf-8').strip()
            print("Received:", line)
        except KeyboardInterrupt:
            print("Serial reading interrupted. Exiting...")
            break

# Close the serial port connection
ser.close()
```

Explanation:

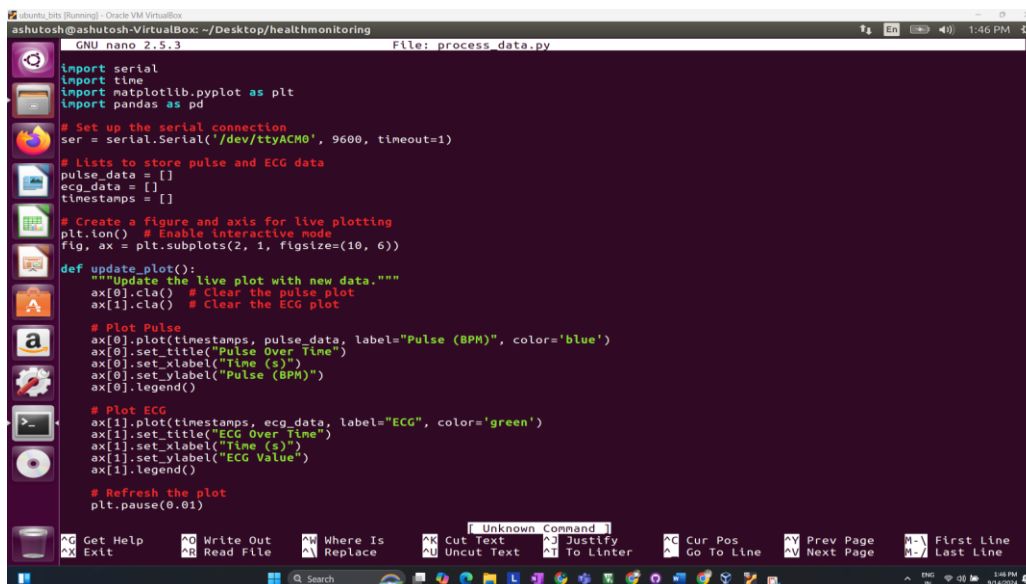
- The `serial.Serial()` function opens the USB serial port.
- The script reads incoming data from the STM32, which is sent every second, and prints it to the console.



```
ashutosh@ashutosh-VirtualBox: ~/Desktop/healthmonitoring
ashutosh@ashutosh-VirtualBox:~/Desktop/healthmonitoring$ sudo nano read_data.py
ashutosh@ashutosh-VirtualBox:~/Desktop/healthmonitoring$ sudo python3 read_data.py
Received: Hello from STM32 for USB Testing setup
Received: Hello from STM32 for USB Testing setup
Received: Hello from STM32 for USB Testing setup
Received: Hello from STM32 for USB Testing setup
Received: Hello from STM32 for USB Testing setup
Received: Hello from STM32 for USB Testing setup
Received: Hello from STM32 for USB Testing setup
```

3: Visualizing the Data (Basic Visualization) (Testing)

Once the data is received from STM32, it is important to visualize it. Using matplotlib, you can plot the ECG and heart pulse data in real-time.



```
GNU nano 2.5.3 File: process_data.py
import serial
import time
import matplotlib.pyplot as plt
import pandas as pd

# Set up the serial connection
ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)

# Lists to store pulse and ECG data
pulse_data = []
ecg_data = []
timestamps = []

# Create a figure and axis for live plotting
plt.ion() # Enable interactive mode
fig, ax = plt.subplots(2, 1, figsize=(10, 6))

def update_plot():
    """Update the live plot with new data."""
    ax[0].cla() # Clear the pulse plot
    ax[1].cla() # Clear the ECG plot

    # Plot Pulse
    ax[0].plot(timestamps, pulse_data, label="Pulse (BPM)", color='blue')
    ax[0].set_title("Pulse Over Time")
    ax[0].set_xlabel("Time (s)")
    ax[0].set_ylabel("Pulse (BPM)")
    ax[0].legend()

    # Plot ECG
    ax[1].plot(timestamps, ecg_data, label="ECG", color='green')
    ax[1].set_title("ECG Over Time")
    ax[1].set_xlabel("Time (s)")
    ax[1].set_ylabel("ECG Value")
    ax[1].legend()

    # Refresh the plot
    plt.pause(0.01)
```

```

GNU nano 2.5.3 File: process_data.py
start_time = time.time()

while True:
    # Read data from the serial port
    try:
        data = ser.readline().decode('utf-8').strip()
        if data:
            # Parse pulse and ECG values from the data string
            parts = data.split(",")
            pulse = int(parts[0].split(":")[1])
            ecg = int(parts[1].split(":")[1])

            # Get the current timestamp
            elapsed_time = round(time.time() - start_time, 2)

            # Append data to the lists
            pulse_data.append(pulse)
            ecg_data.append(ecg)
            timestamps.append(elapsed_time)

            # Update the plot
            update_plot()

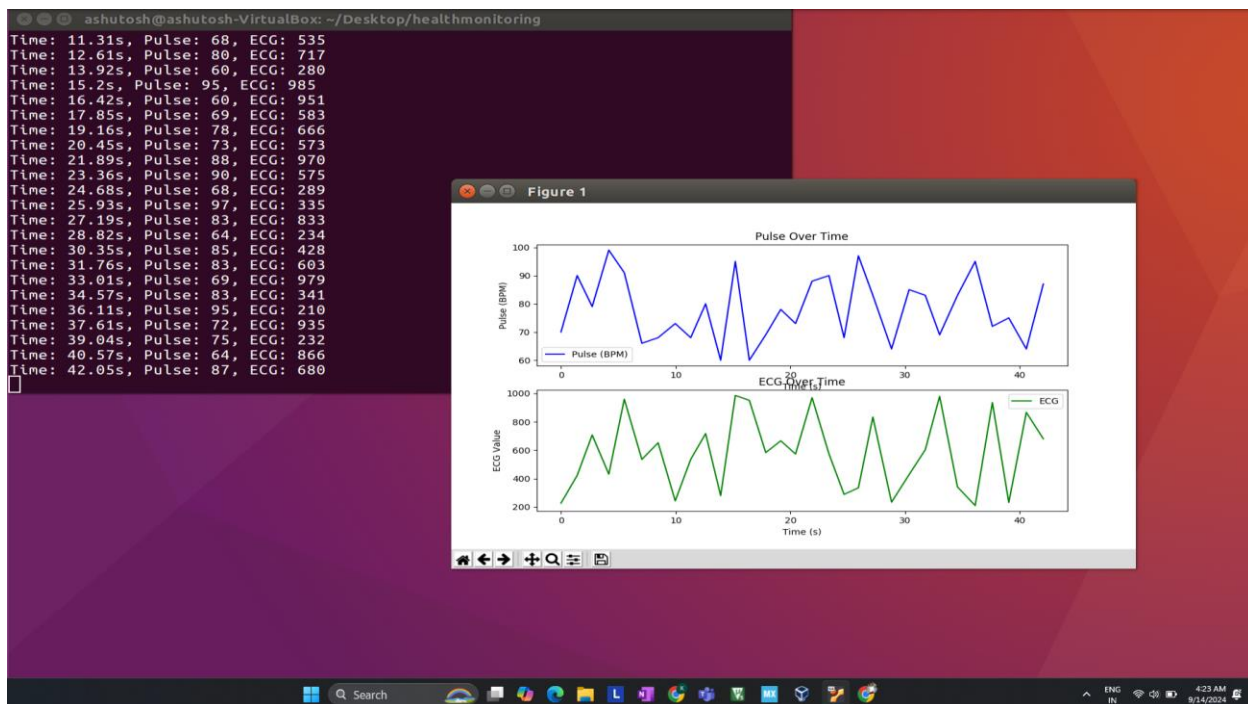
            # Print the received data for logging
            print("Time: {}s, Pulse: {}, ECG: {}".format(elapsed_time, pulse, ecg))

            # Add a small delay
            time.sleep(1)
    except KeyboardInterrupt:
        # Save the data to a CSV file on exit
        df = pd.DataFrame({
            "Time (s)": timestamps,
            "Pulse (BPM)": pulse_data,
            "ECG": ecg_data
        })
        df.to_csv("sensor_data.csv", index=False)
        print("Data saved to sensor_data.csv.")
        break

Get Help  Write Out  Where Is  Cut Text  Justify  Cur Pos  Prev Page  First Line
Exit      Read File  Replace  Uncut Text  To Linter  Go To Line  Next Page  Last Line

```

Result:

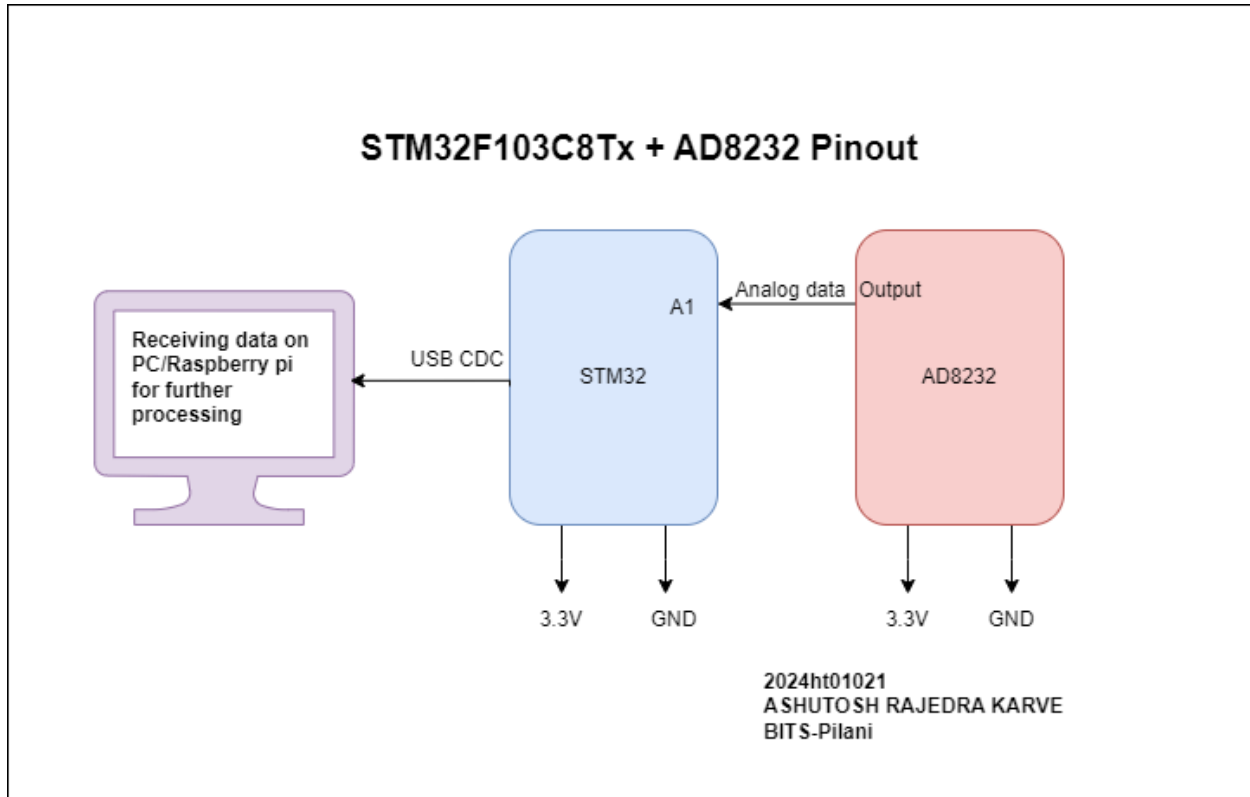


Real-time ECG Plot Testing...

Once the raw data is captured, you can move on to basic data processing and visualization to analyze ECG signals and detect abnormalities.

5 Setting up actual sensor (ECG Sensor) & Final processing code

Pin-Configuration Block Diagram of STM32 + AD8232



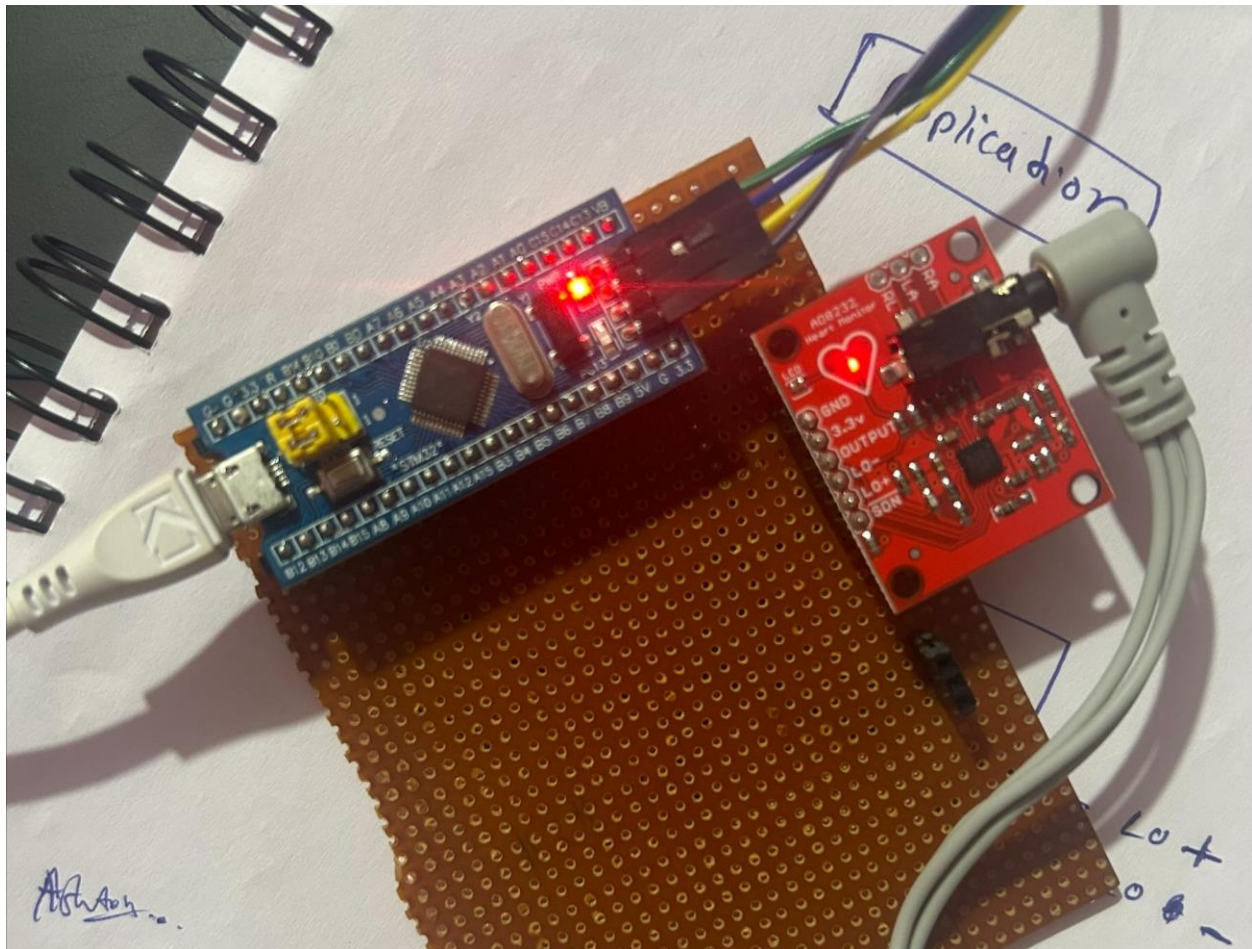
Code//

```
main.c
100 // USER CODE BEGIN 2
101
102 // Infinite loop
103 // USER CODE BEGIN WHILE
104 while (1)
105 {
106 // Infinite loop
107 // USER CODE BEGIN WHILE
108 while (1)
109 {
110 // Start ADC conversion
111 HAL_ADC_Start(&hadc1);
112
113 // Poll for ADC conversion completion
114 if (HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY) == HAL_OK)
115 {
116 // Get the ADC value from PA1 (A1)
117 uint32_t adcValue = HAL_ADC_GetValue(&hadc1);
118
119 // Convert the ADC value to a string
120 char buffer[10];
121 sprintf(buffer, "%i\n", adcValue); // Send raw value
122
123 // Transmit the data over USB
124 CDC_Transmit_FS((uint8_t*)buffer, strlen(buffer));
125 }
126
127 // Stop the ADC
128 HAL_ADC_Stop(&hadc1);
129
130 // Delay adjusted for higher sampling rate (2ms for 500Hz sampling rate)
131 HAL_Delay(2);
132 }
133 }
134 // USER CODE END WHILE
135 }
```

Build Output

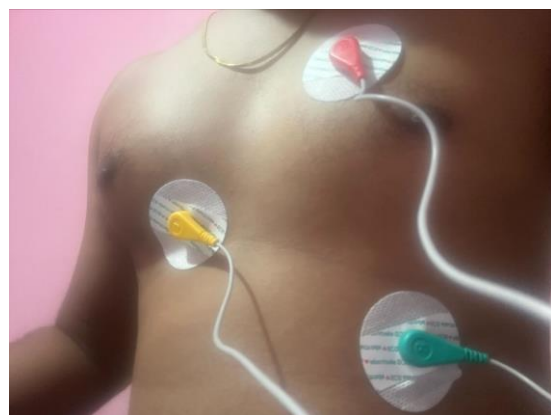
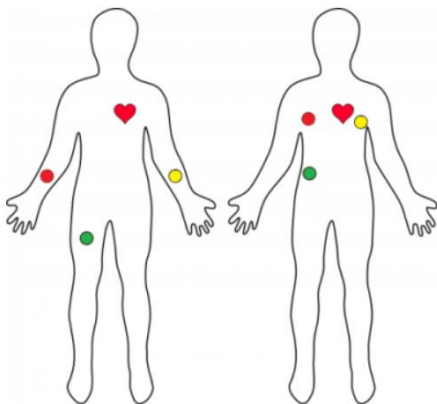
```
Program Size: Code=15368 RO-data=320 RW-data=388 ZI-data=4292
FromIcF: creating hex file...
"HealthMonitor\HealthMonitor.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:14
Load "HealthMonitor\HealthMonitor.axf"
Erase Done.
Programming Done.
Verify OK.
Application running ...
Flash Load finished at 23:25:03
```

Actual Hardware setup



ECG Sensor Placement on Body

It is recommended to snap the sensor pads on the leads before sticking to the body, the closer to the heart, the better the measurement would be. The cables are color-coded to help you identify a proper placement.



Before moving Ahead Let's understand what is ECG?

ECG is the abbreviated term for an electrocardiogram. It is used to record the electrical activity of the heart from different angles to both identify and locate pathology. Electrodes are placed on different parts of a patient's limbs and chest to record the electrical activity.

Parts of the ECG explained

- **P waves**

P waves represent **atrial depolarization**

In healthy individuals, there should be a **P wave** preceding each **QRS** complex.

- **PR interval**

The **PR interval** begins at the **start of the P wave** and ends at **the beginning** of the **Q wave**. It **represents** the **time** for electrical activity to move **between the atria and the ventricles**.

- **QRS complex**

The **QRS complex** represents the **depolarisation of the ventricles**.

It appears as three closely related waves on the ECG (the Q, R and S wave).

- **ST segment**

The **ST segment starts** at the **end** of the **S wave** and **ends** at the **beginning** of the **T wave**.

The **ST segment** is an isoelectric line representing the time **between depolarisation and repolarisation** of the **ventricles** (i.e. ventricular contraction).

- **T wave**

The **T wave** represents **ventricular repolarisation**.

It appears as a small wave **after** the **QRS complex**.

- **RR interval**

The **RR interval begins** at the **peak of one R wave** & ends at the **peak of the next R wave**.

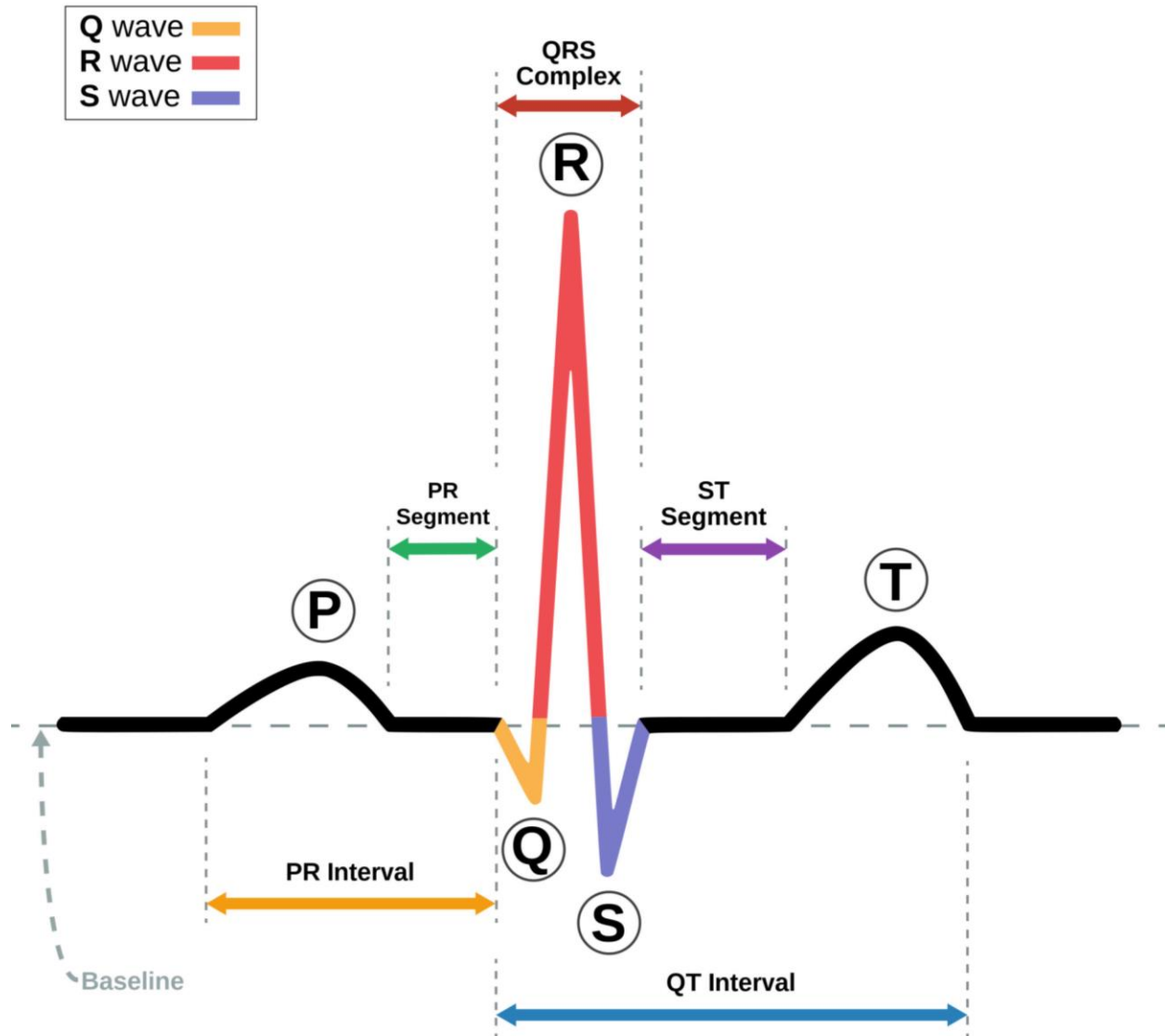
It **represents** the **time between two QRS complexes**.

- **QT interval**

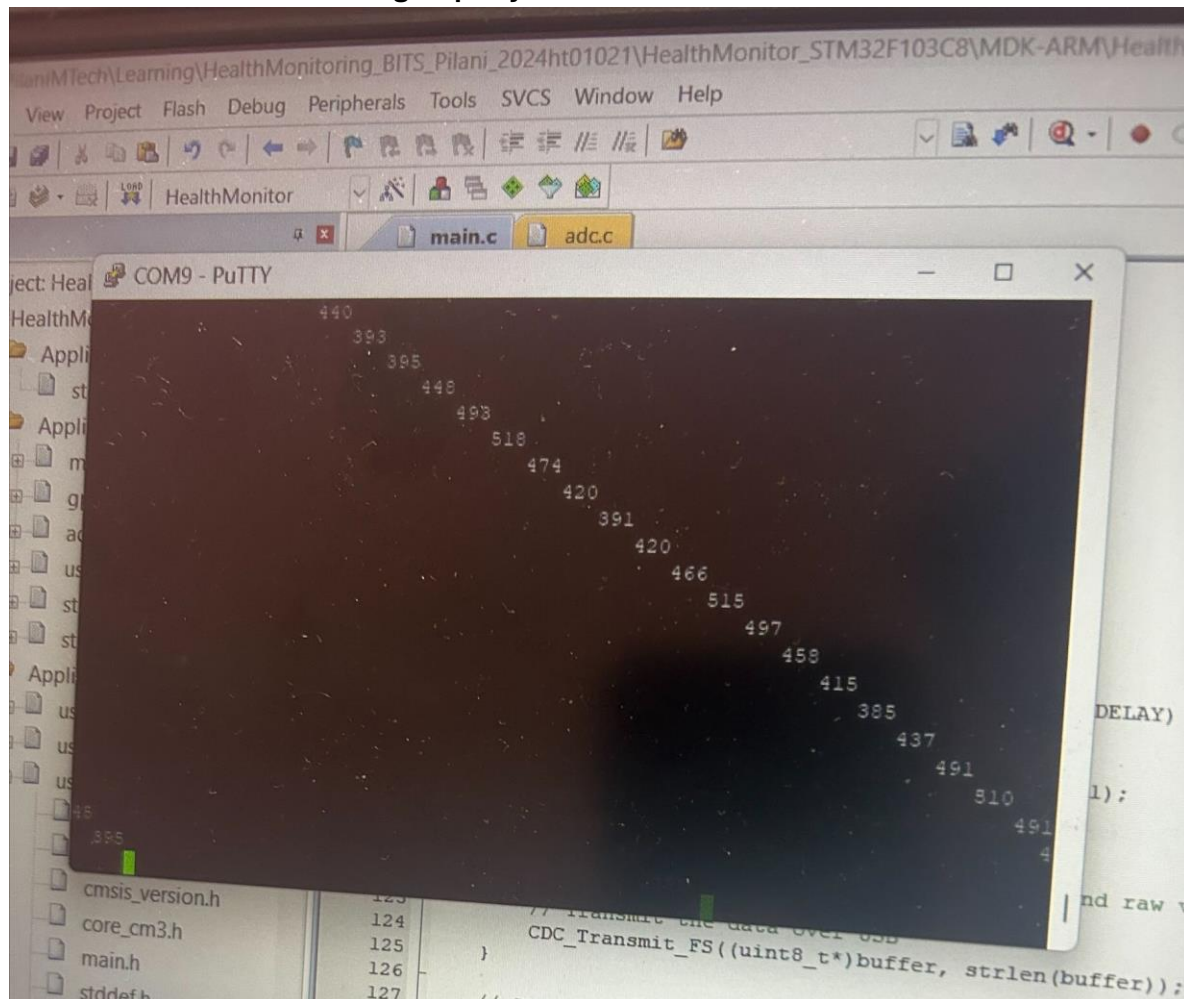
The **QT interval** begins at the start of the **QRS complex** & finishes at the end of the **T wave**.

It **represents** the time taken for the **ventricles** to **depolarise** and then **repolarise**.

Parts of the ECG



Raw data which I am receiving on putty



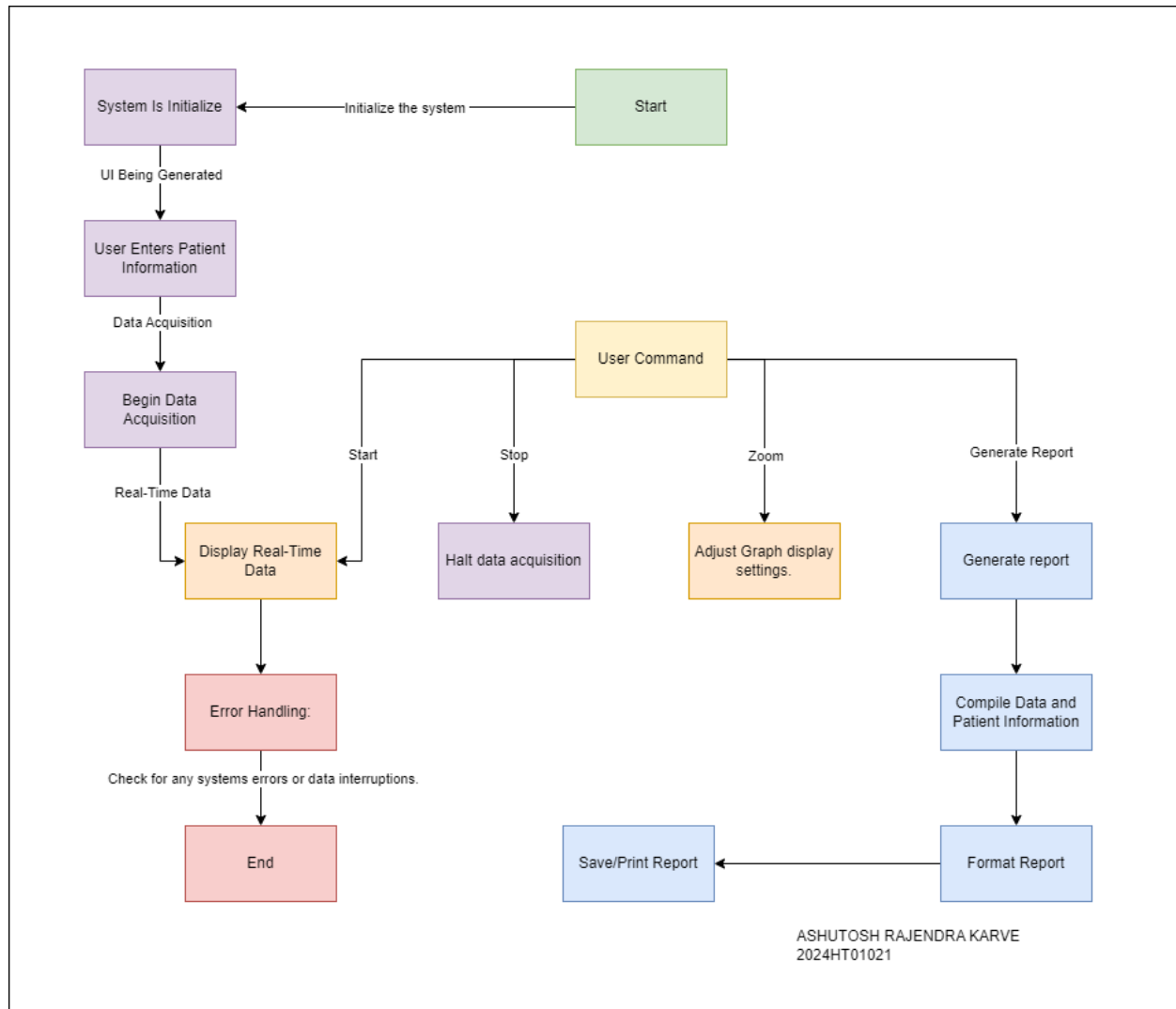
Observations from Putty Display of ECG Data

Interpreting the Data:

- **Fluctuations in the range of 390–530:** The readings captured in the Putty terminal indicate fluctuations between 390 and 530. These fluctuations represent various components of the ECG signal, primarily depicting the heart's electrical activity.
- **QRS Complex:** Values near the higher end of this range, particularly those approaching 530, are likely associated with the QRS complex, indicating ventricular contraction. This is the most prominent and critical part of an ECG trace.
- **P-Wave and T-Wave:** Lower values in this range, closer to 390, could potentially represent the P-wave or T-wave, which are less pronounced than the QRS complex and signify atrial depolarization and ventricular repolarization, respectively.

Periodic Patterns: The data appears to be periodically fluctuating, consistent with the expected rhythmic pattern of a normal ECG, indicating the regular beating of the heart. This rhythmic pattern is essential for analyzing cardiac function and identifying potential abnormalities.

Software Architecture for Data Processing (FLOW CHART)



Data Acquisition: Continuous reading and real-time display of ECG data.

User Interaction: Users can start/stop data acquisition, adjust graph settings, or initiate report generation.

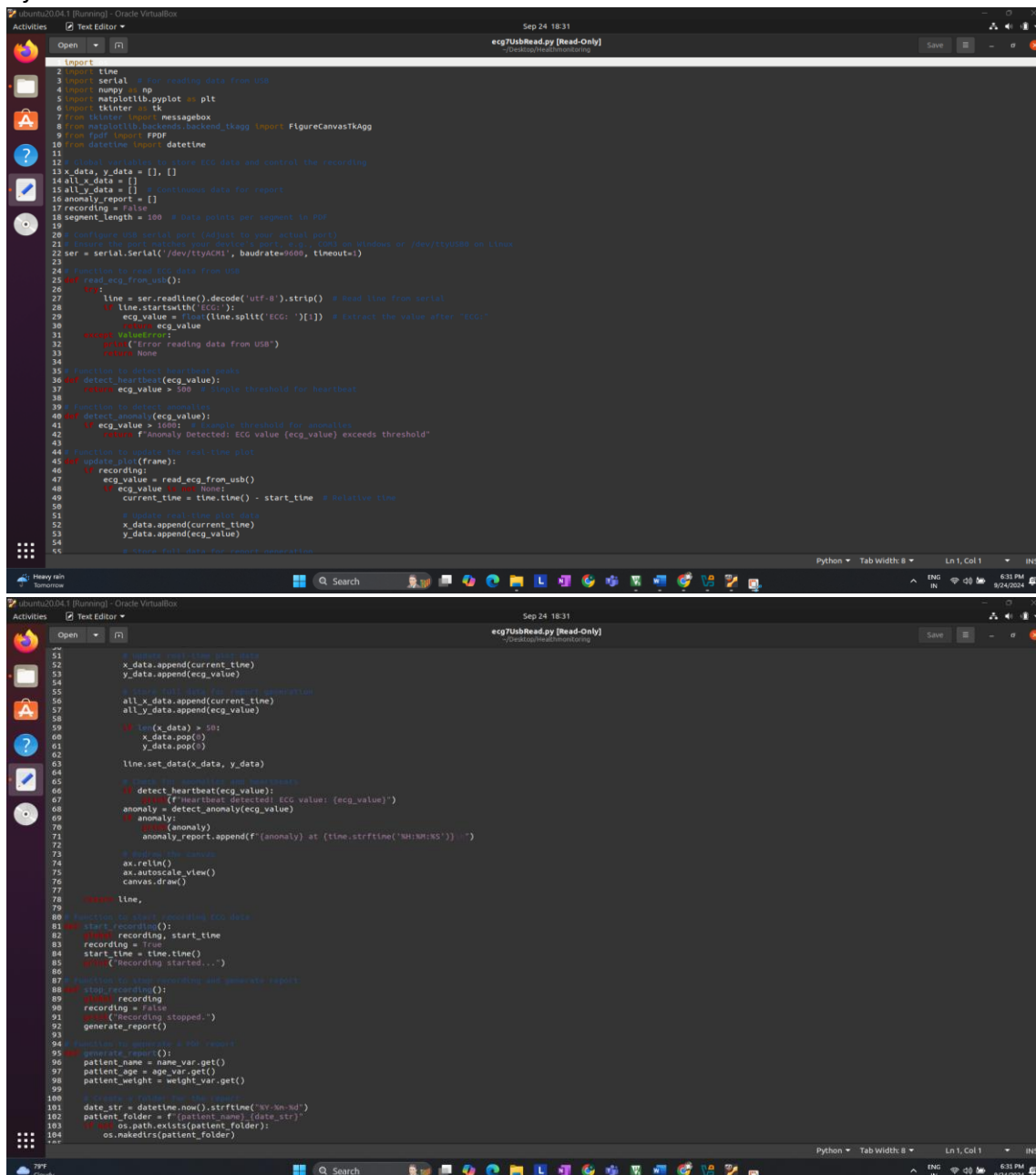
Report Generation: Compiles data and patient information, formats it, and allows for saving or printing.

Python Program for further processing and report generation (Enhanced) (FINAL)

Install the following libraries which are crucial for data handling, real-time plotting, and report generation:

- **matplotlib** for data visualization.
- **numpy** for numerical operations.
- **pyserial** for communication with the STM32 device over USB.
- **fpdf** or **reportlab** for generating PDF reports.

Python Code>>



```
1 # Import necessary libraries
2 import time
3 import serial
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import tkinter as tk
7 from tkinter import messagebox
8 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
9 from fpdf import FPDF
10 from datetime import datetime
11
12 # Global variables to store ECG data and control the recording
13 x_data, y_data = [], []
14 all_x_data = []
15 all_y_data = [] # Continuous data for report
16 anomaly_report = []
17 recording = False
18 segment_length = 100 # Data points per segment in FPDF
19
20 # Configure USB serial port (Adjust to your actual port)
21 # ser = serial.Serial('/dev/ttyACM0', baudrate=9600, timeout=1)
22 ser = serial.Serial('/dev/ttyACM1', baudrate=9600, timeout=1)
23
24 # Function to read ECG data from USB
25 def read_ecg_from_usb():
26     try:
27         line = ser.readline().decode('utf-8').strip() # Read line from serial
28         if line.startswith('ECG:'):
29             ecg_value = float(line.split('ECG: ')[1]) # Extract the value after 'ECG:'
30             return ecg_value
31         else:
32             raise ValueError("Invalid ECG data format")
33     except (ValueError, serial.SerialException):
34         return None
35
36 # Function to detect heartbeat peaks
37 def detect_heartbeat(ecg_value):
38     return ecg_value > 500 # Simple threshold for heartbeat
39
40 # Function to detect anomalies
41 def detect_anomaly(ecg_value):
42     if ecg_value > 1000: # Anomaly threshold for anomalies
43         return f"Anomaly Detected: ECG value (ecg_value) exceeds threshold"
44     return None
45
46 # Function to update the real-time plot
47 def update_plot(frame):
48     if recording:
49         ecg_value = read_ecg_from_usb()
50         if ecg_value is not None:
51             current_time = time.time() - start_time # Relative time
52             # Update real-time plot data
53             x_data.append(current_time)
54             y_data.append(ecg_value)
55
56 # Update plot data for report generation
57 all_x_data.append(current_time)
58 all_y_data.append(ecg_value)
59
60 if len(x_data) > 50:
61     x_data.pop(0)
62     y_data.pop(0)
63     line.set_data(x_data, y_data)
64
65 # Detect heartbeat and anomalies
66 if detect_heartbeat(ecg_value):
67     heartbeat_detected = True
68     anomaly = detect_anomaly(ecg_value)
69     anomaly_report.append(anomaly)
70     anomaly_report.append(f"(anomaly) at {time.strftime('%H:%M:%S')}")
71
72 # Refresh the plot
73 ax.relim()
74 ax.autoscale_view()
75 canvas.draw()
76
77 # Update line
78
79 # Function to start recording and generate report
80 def start_recording():
81     global recording, start_time
82     recording = True
83     start_time = time.time()
84     messagebox.showinfo("Recording started...")
85
86 # Function to stop recording and generate report
87 def stop_recording():
88     global recording
89     recording = False
90     messagebox.showinfo("Recording stopped.")
91     generate_report()
92
93 # Function to generate a PDF report
94 def generate_report():
95     global patient_name, patient_age, patient_weight, date_str
96     patient_name = name_var.get()
97     patient_age = age_var.get()
98     patient_weight = weight_var.get()
99
100 # Create a folder for the report
101 date_str = datetime.now().strftime("%Y-%m-%d")
102 patient_folder = f"{patient_name}_{date_str}"
103 if not os.path.exists(patient_folder):
104     os.makedirs(patient_folder)
```

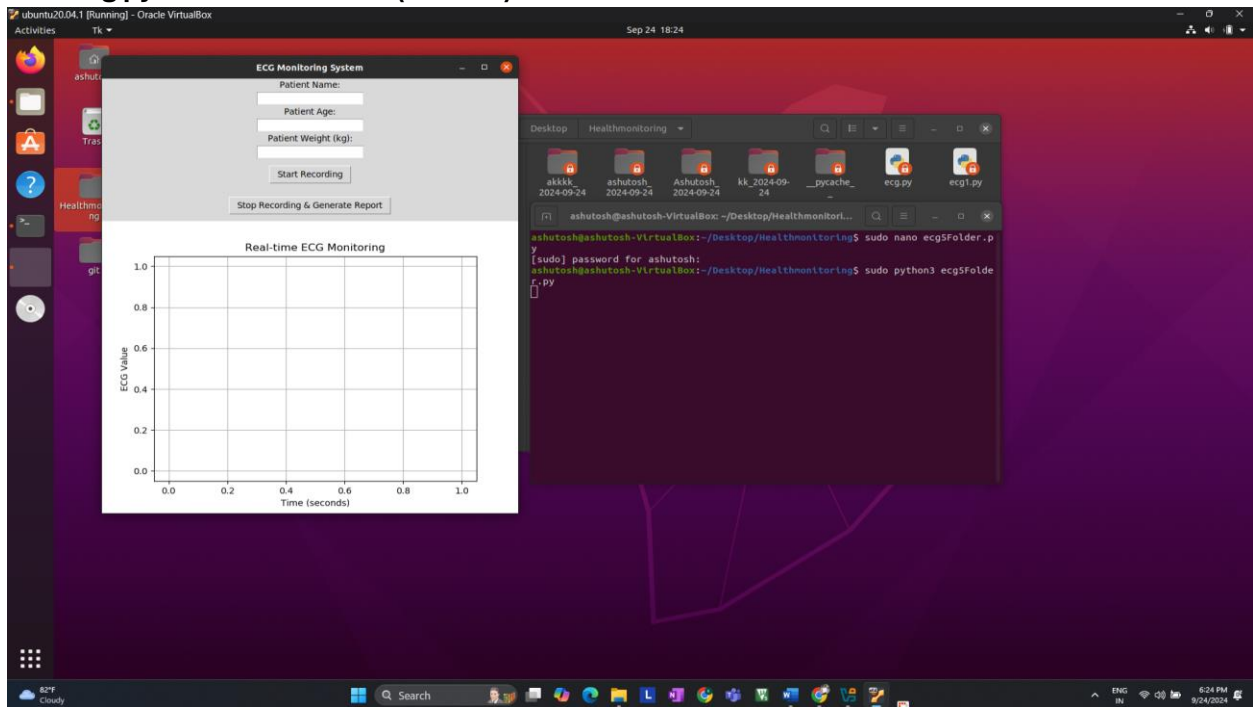
```
ubuntu20.04.1 [Running] - Oracle VM VirtualBox
Activities Text Editor
Sep 24 18:31
ecg7UsbRead.py [Read-Only]
~/Desktop/HeartRateMonitoring

101 date_str = datetime.now().strftime("%Y-%m-%d")
102 patient_folder = f"patient_{name}_{date_str}"
103 os.path.exists(patient_folder):
104     os.makedirs(patient_folder)
105
106 # filename for ECG report
107 report_filename = os.path.join(patient_folder, f"{patient_name}_ECG_Report.pdf")
108
109 # create PDF
110 pdf = FPDF()
111 pdf.add_page()
112
113 # Add patient info to the PDF
114 pdf.set_font('Arial', size=12)
115 pdf.cell(200, 10, txt="ECG Anomaly Detection Report", ln=True, align="C")
116 pdf.cell(200, 10, txt=f"Patient Name: {patient_name}", ln=True, align="L")
117 pdf.cell(200, 10, txt=f"Patient Age: {patient_age}", ln=True, align="L")
118 pdf.cell(200, 10, txt=f"Patient Weight: {patient_weight}", ln=True, align="L")
119 pdf.cell(200, 10, txt=f"Date: {time.strftime('%Y-%m-%d')}", ln=True, align="L")
120 pdf.cell(200, 10, txt=f"Total anomalies detected: {len(anomaly_report)}", ln=True, align="L")
121
122 # Split the ECG data segments and add each segment to the PDF
123 total_points = len(all_x_data)
124 num_segments = total_points // segment_length
125
126 y_offset = 0 # y offset for the y position of each segment in the PDF
127
128 for i in range(num_segments + 1):
129     start_idx = i * segment_length
130     end_idx = min(start_idx + segment_length, total_points)
131
132     # start_idx < end_idx:
133     # Create a figure for each segment
134     fig, segment_ax = plt.subplots(figsize=(8, 2))
135     segment_ax.plot(all_x_data[start_idx:end_idx], all_y_data[start_idx:end_idx], color='blue', lw=1)
136     segment_ax.set_title(f"ECG Waveform Segment {i + 1}")
137     segment_ax.set_xlabel("Time (seconds)")
138     segment_ax.set_ylabel("ECG Value")
139     segment_ax.grid(True)
140
141 # Save the segment as an image
142 segment_filename = os.path.join(patient_folder, f"ecg_segment_{i + 1}.png")
143 fig.savefig(segment_filename, bbox_inches='tight')
144
145 # Add the segment image to the PDF
146 pdf.image(segment_filename, x=10, y=y_offset, w=100)
147 y_offset += 65 # Add the y offset for the next segment
148
149 # Add a new page if the y offset exceeds page height
150 if y_offset > 270:
151     pdf.add_page()
152     y_offset = 0
153
154 # Add the conclusion to the PDF
```

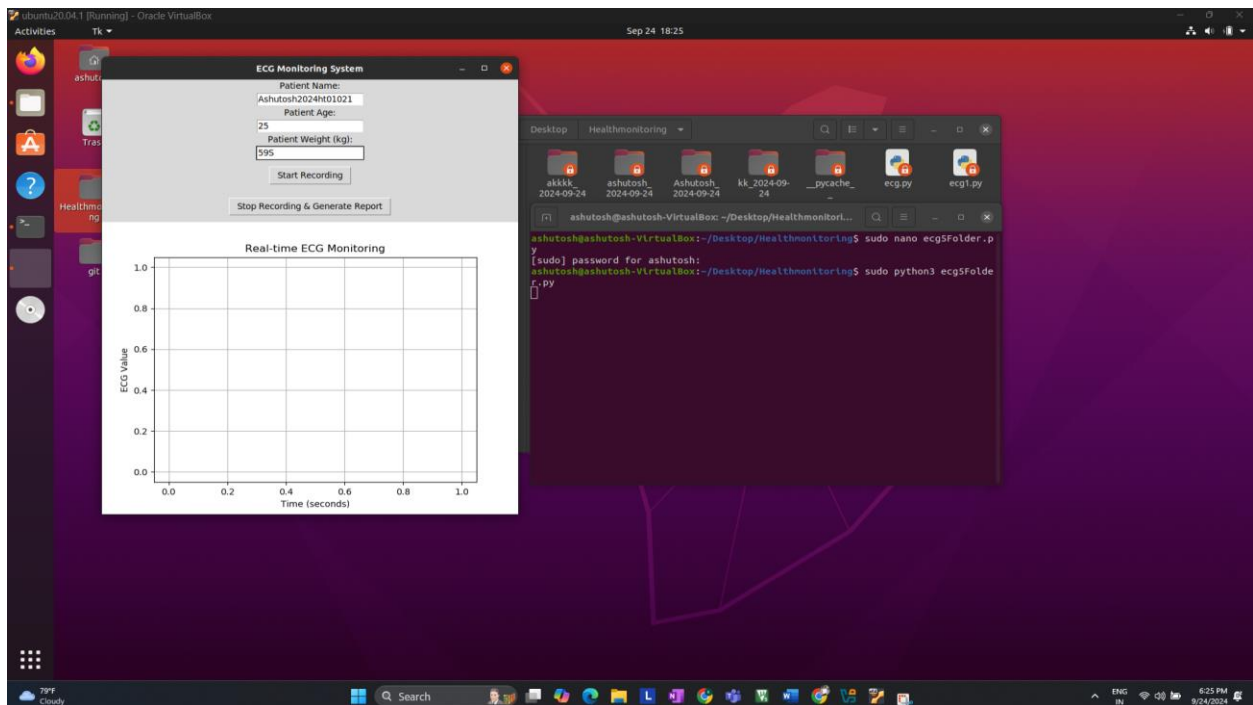
```
ubuntu20.04.1 [Running] - Oracle VM VirtualBox
Activities Text Editor
Sep 24 18:32
ecg7UsbRead.py [Read-Only]
~/Desktop/HeartRateMonitoring

160 pdf.set_font('Arial', size=12, style='B')
161 pdf.cell(200, 10, txt="Conclusion", ln=True, align="L")
162 pdf.set_font('Arial', size=10)
163 conclusion = f"The ECG data indicates {len(anomaly_report)} anomalies. Further medical review is recommended."
164 pdf.multi_cell(0, 10, conclusion)
165
166 # Save the PDF
167 pdf.output(report_filename)
168 # Report saved: {report_filename}
169 messagebox.showinfo("Report", f"ECG report saved as {report_filename}")
170
171 # Create the Tkinter GUI
172 root = tk.Tk()
173 root.title("ECG Monitoring System")
174
175 # Add labels for patient info
176
177 age_var = tk.StringVar()
178 weight_var = tk.StringVar()
179
180 tk.Label(root, text="Patient Name:").pack()
181 tk.Entry(root, textvariable=name_var).pack()
182
183 tk.Label(root, text="Patient Age:").pack()
184 tk.Entry(root, textvariable=age_var).pack()
185
186 tk.Label(root, text="Patient Weight (kg):").pack()
187 tk.Entry(root, textvariable=weight_var).pack()
188
189 # Add a button to start recording
190 start_button = tk.Button(root, text="Start Recording", command=start_recording)
191 start_button.pack(pady=10)
192
193 stop_button = tk.Button(root, text="Stop Recording & Generate Report", command=stop_recording)
194 stop_button.pack(pady=10)
195
196 # Set up the Matplotlib figure and axes for the real-time ECG plot
197 fig, ax = plt.subplots()
198 line, = ax.plot([], [], label='ECG Data', color='blue')
199
200 ax.set_title("Real-time ECG Monitoring")
201 ax.set_xlabel("Time (seconds)")
202 ax.set_ylabel("ECG Value")
203 ax.grid(True)
204
205 # Embed the plot in the Tkinter window
206 canvas = FigureCanvasTkAgg(fig, master=root)
207 canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
208
209 # Animation to update the plot with incoming data
210 from matplotlib.animation import FuncAnimation
211 ani = FuncAnimation(fig, update_plot, interval=200)
212
213 # Main loop
214 root.mainloop()
```

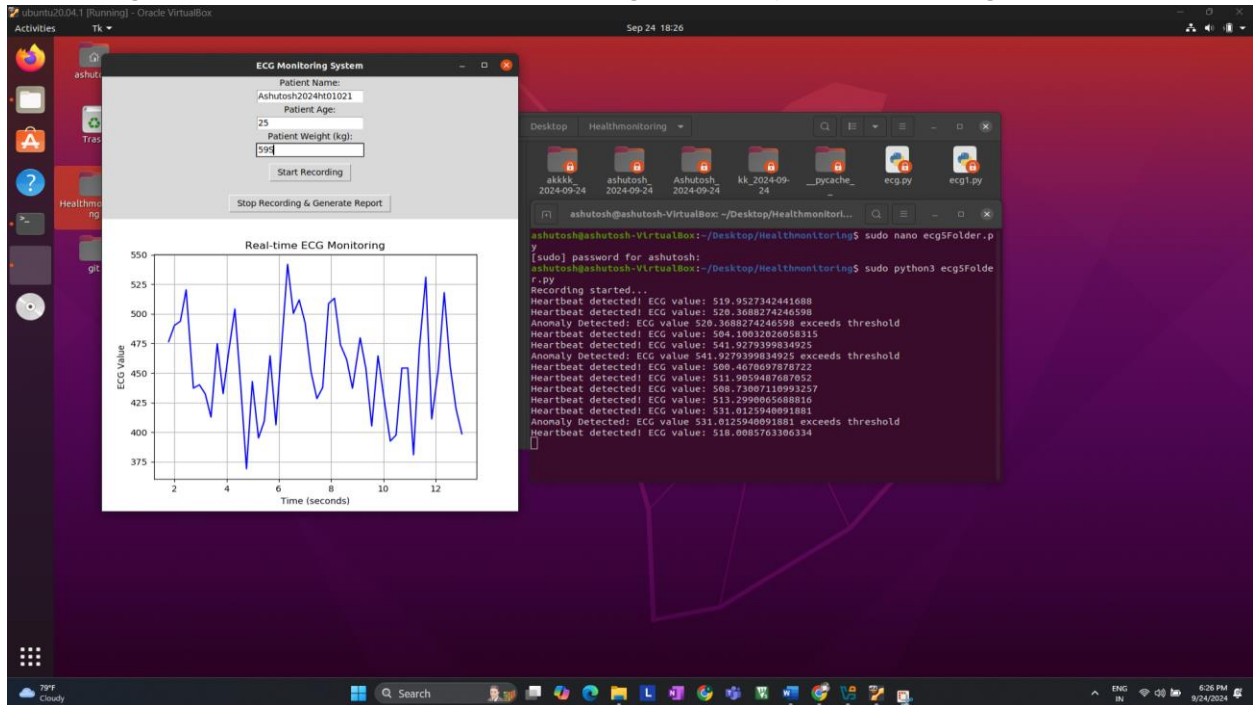
Running python code in Linux(Ubuntu) Machine



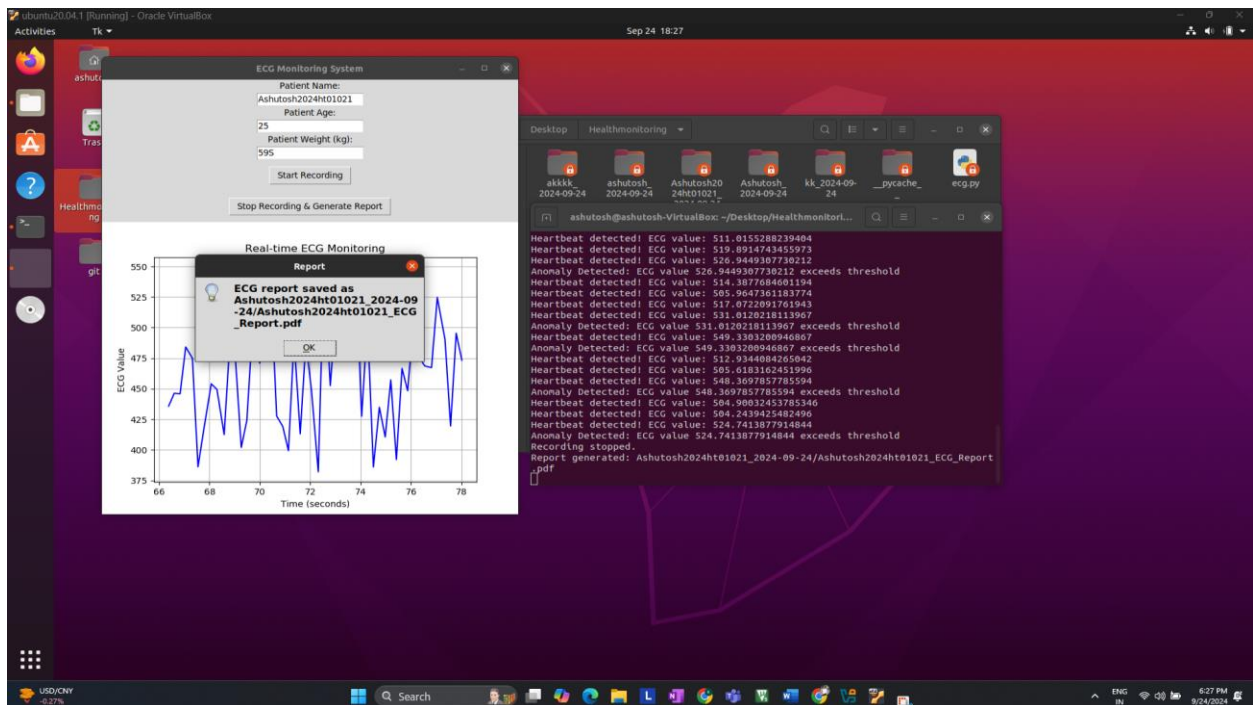
After Running the code, Fill the container with the patient/user details.



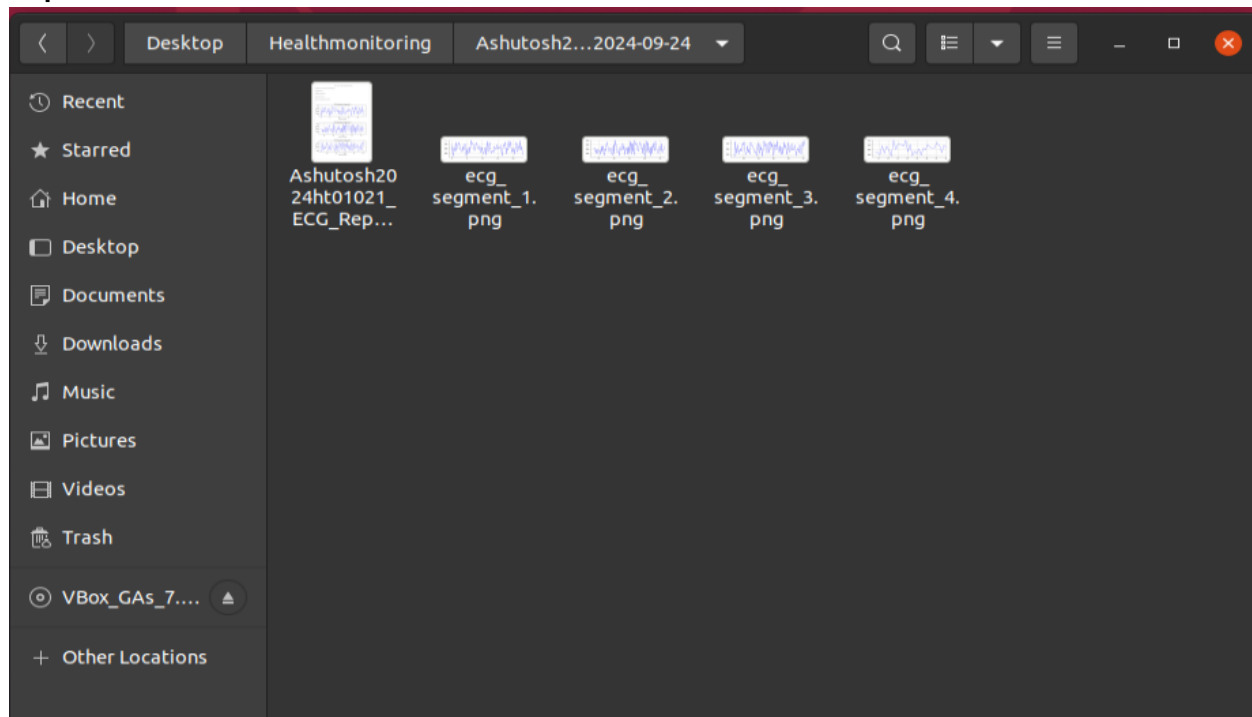
After filling the details and ECG Diodes connecting to the body **start Recording**



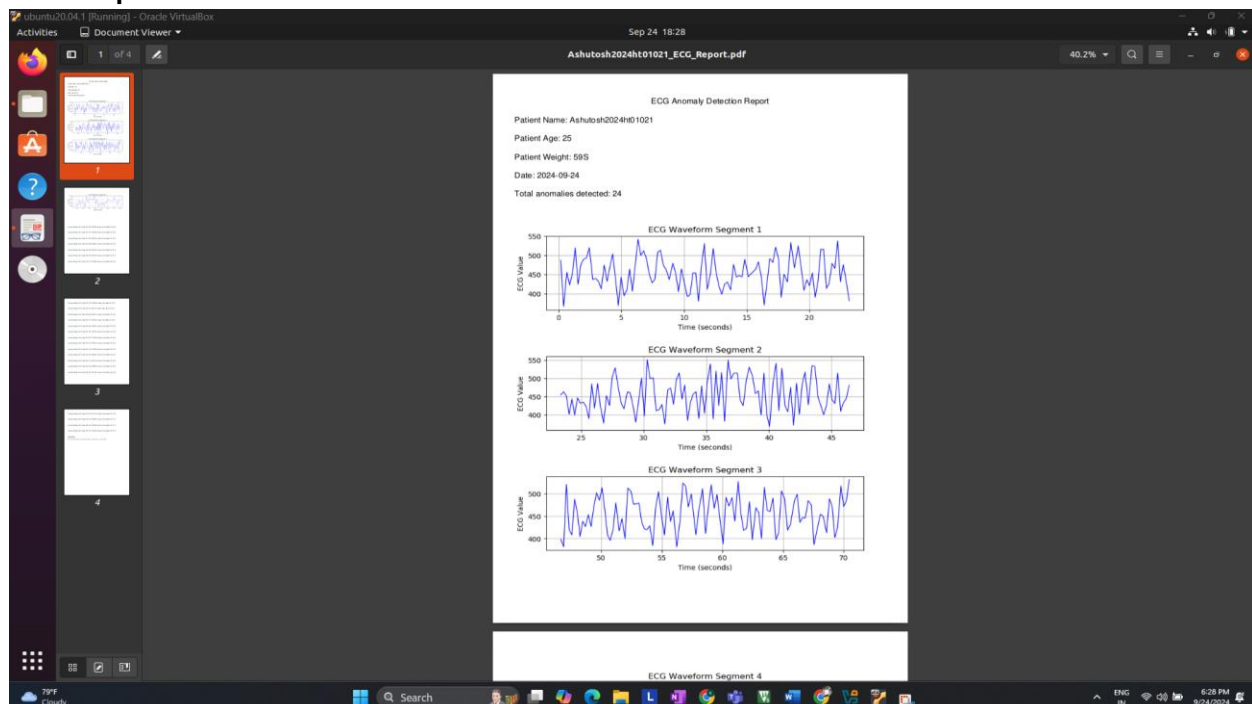
Click **Stop Recording to Generate Report**, then you will ask to save report, click **OK**

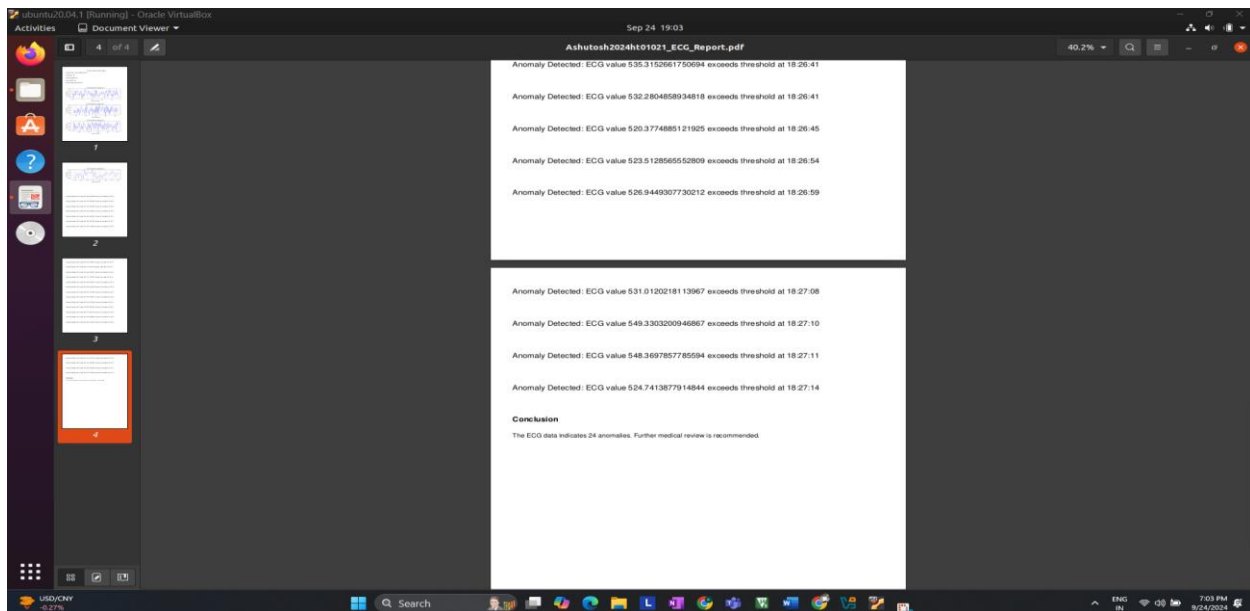


Report will be **saved** on same location in **New Folder** with Patient/User name.



Final Report





6 Challenges and Solutions

Sensor Noise:

One of the major challenges encountered during the development of the ECG monitoring system is the presence of noise and motion artifacts in the ECG signal. These interferences can distort the true heart signal and arise from various sources such as muscle contractions, electrode movement, or external electrical noise. Ensuring proper electrode placement and maintaining steady contact with the skin is essential to mitigate this issue. However, in future versions of the system, software-based filtering will be incorporated to further reduce noise and improve signal quality.

Raw Data Accuracy:

Currently, the raw data captured from the AD8232 ECG sensor is transmitted as unprocessed ADC values. While this provides a direct representation of the analog signal, it does not correspond to standard voltage units (mV) commonly used in ECG analysis. As a result, the raw ADC values can be difficult to interpret without proper scaling. Moving forward, converting these values into meaningful voltage levels will be necessary to facilitate better analysis and comparison of ECG signals.

Data Filtering (Future Plan):

To enhance the quality of the ECG signal, future updates to the system will include the implementation of a smoothing algorithm such as a moving average filter. This filter will help in reducing short-term fluctuations and noise, resulting in a cleaner and more accurate signal for analysis. Additionally, the raw ADC values will be scaled to represent the actual voltages in millivolts (mV), which will make the data more interpretable and allow for easier identification of key ECG components, such as the P-wave, QRS complex, and T-wave.

7 Future Scope

AI-based ECG Analysis:

In future iterations of the system, AI models could be integrated to automatically analyze ECG signals and detect abnormalities like arrhythmias, ischemia, and other heart conditions. Machine learning algorithms, such as convolutional neural networks (CNNs), can be trained on large datasets of ECG signals to identify these irregularities. This would allow for real-time diagnostic feedback, potentially alerting users to abnormal heart rhythms or conditions that require medical attention.

Pulse Sensor Integration:

Another enhancement is the integration of a **Heartbeat Pulse Sensor** to provide additional heart rate monitoring through photoplethysmography (PPG). The combination of both ECG and pulse data would offer a more comprehensive view of cardiovascular health. By analyzing the synchronization between the pulse and ECG signals, future versions of the system could improve the detection of arrhythmias and other irregularities.

Advanced Data Logging:

Future versions will support comprehensive data logging, enabling the storage of ECG signals over time. This would allow for detailed analysis of a patient's heart activity across multiple sessions. The system will generate reports containing graphs, patient information (e.g., name, age, weight), and any detected abnormalities. This data could be stored locally or in the cloud for remote monitoring by healthcare providers.

Wireless Communication:

To enhance portability and usability, future updates may include wireless communication using Bluetooth or Wi-Fi. This will allow the ECG data to be transmitted wirelessly to a smartphone or cloud platform, enabling real-time monitoring without the need for wired connections. This feature would make the system more convenient for patients and medical professionals.

8 Conclusion

Current Status:

The **Advance Health Monitoring System** has successfully demonstrated real-time ECG signal acquisition and basic visualization. The STM32 Blue Pill microcontroller captures raw ECG data from the AD8232 sensor, which is then transmitted over USB and processed using Python. Initial tests show promising results in capturing and displaying ECG waveforms.

Next Steps:

Future development will focus on improving the signal quality through data filtering and scaling, integrating a pulse sensor, and adding advanced features like AI-based ECG analysis. This will allow the system to detect heart abnormalities automatically. Additionally, the system will evolve to include wireless communication for better portability and ease of use.

Impact:

This project has the potential to develop into a low-cost, portable, and comprehensive health monitoring system. It can be used to assist in early detection of heart issues, helping individuals and healthcare providers track cardiovascular health more effectively. By continuously expanding its functionality, the system could become a valuable tool for non-invasive heart monitoring in both clinical and personal health settings.

9 References

1. STMicroelectronics. STM32 Blue Pill Documentation: <https://www.st.com/en/microcontrollers-microprocessors/stm32-bluepill.html>
2. Analog Devices. AD8232 Single-Lead Heart Rate Monitor Front End: <https://www.analog.com/en/products/ad8232.html>
3. Python Software Foundation. Python Documentation: <https://docs.python.org/3/>
4. PySerial Library Documentation: <https://pyserial.readthedocs.io/en/latest/>
5. Matplotlib Library Documentation: <https://matplotlib.org/stable/index.html>
6. Heart Rate Variability and ECG Analysis using AI, IEEE Conference Paper: <https://ieeexplore.ieee.org/document/9173269>
7. "Photoplethysmography for Pulse Detection in Wearable Devices," Journal of Biomedical Engineering: <https://www.jbiomecheng.org/ppg-sensors>

GITHUB PROJECT LINK:

https://github.com/Ashutoshkarve007/HealthMonitoring_BITS_Pilani_2024ht01021.git

GITHUB PROFILE:

<https://github.com/Ashutoshkarve007>

LINKDIN LINK:

<https://in.linkedin.com/in/ashutoshkarve>

MY YOUTUBE CHANNEL:

<https://www.youtube.com/@ashutoshkarve698>